Subject

Object Oriented programming

and

Computer Graphics

Unit-III

Inheritance and Polymorphism
Exception Handling and
Multithreading

Contents

- **Inheritance:** Super classes and Subclasses, protected members, relationship between super classes and subclasses, types of Inheritance, constructors in subclasses, object class.

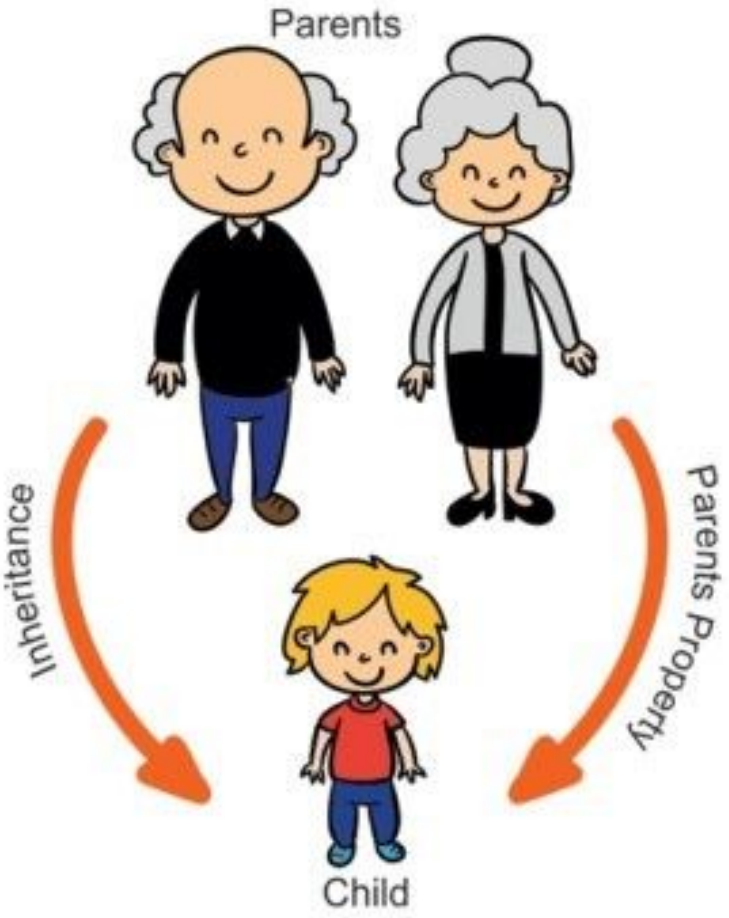
- **Polymorphism:** Abstract classes and methods, final methods and classes, dynamic binding, polymorphism examples and Interfaces.

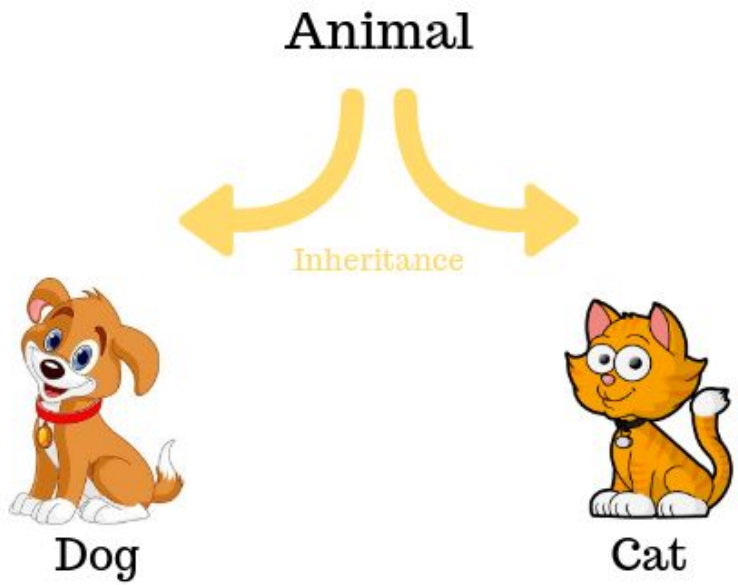
- **Exception handling:** fundamentals, Exception Types, Using try-catch, Multiple try-catch clauses, Nested try statements, throw, throws, finally, Built-in Exceptions

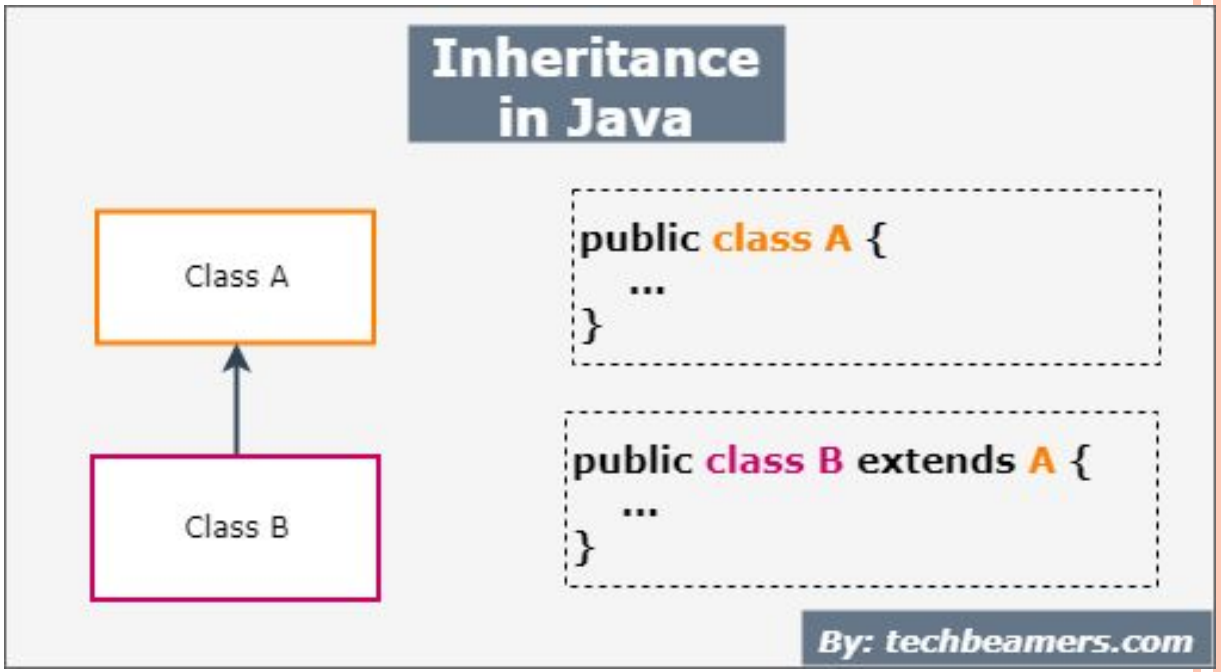
- **MultiThreading:** Java Thread Model, Main Thread, Creating a Thread , Creating Multiple Threads.

- **Case Study :** Demonstrate online Banking/Library system using Inheritance, Exception handling and Multi-Threading concepts

Inheritance




Animal
Inheritance
Dog
Cat


Inheritance in Java
Class A
Class B
public class A {
...
}
public class B extends A {
...
}
By: techbeamers.com

INHERITANCE:

- It is the mechanism in Java by which one class is allowed to inherit the features(fields and methods) of another class.
- In Java, Inheritance means creating new classes based on existing ones.
- A class that inherits from another class can reuse the methods and fields of that class.
- In addition, you can add new fields and methods to your current class as well.
- **Syntax**

```
class  ParentClass {

   //fields and methods
}
class ChildClass extends ParentClass {

   // Additional fields and methods
}
```

- **Key Terminologies Used in Java Inheritance**

- **Super Class/ Parent Class/ Base Class:** The class whose features are inherited is known as a superclass(or a base class or a parent class).

- **Sub Class/ Child Class/ Derived Class:** The class that inherits the other class is known as a subclass(or a derived class, extended class or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.

- **extends Keyword:** This keyword is used to inherit properties from a superclass.

Why Use Inheritance in Java?

- **Code Reusability:** The code written in the Superclass is common to all subclasses. Child classes can directly use the parent class code.
- **Method Overriding:** Method Overriding is achievable only through Inheritance. It is one of the ways by which Java achieves Run Time Polymorphism.
- **Abstraction:** The concept of abstraction where we do not have to provide all details, is achieved through inheritance. Abstraction only shows the functionality to the user.
- ***Note: The extends keyword establishes an "is-a" relationship between the child class and the parent class. This allows a child class to have all the behavior of the parent class.***

```
// Superclass
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

// Subclass
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

public class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();  // inherited method
        d.bark();   // own method
    }
}
```

Protected Members

- The **protected** access modifier allows members to be accessed in the **same package** and by **subclasses** in **same or other packages.**
- **Example:**

```
class Animal {
    protected String type = "Animal";
}

class Dog extends Animal {
    void printType() {
        System.out.println("Type: " + type); // Accessible due to protected
    }
}
```

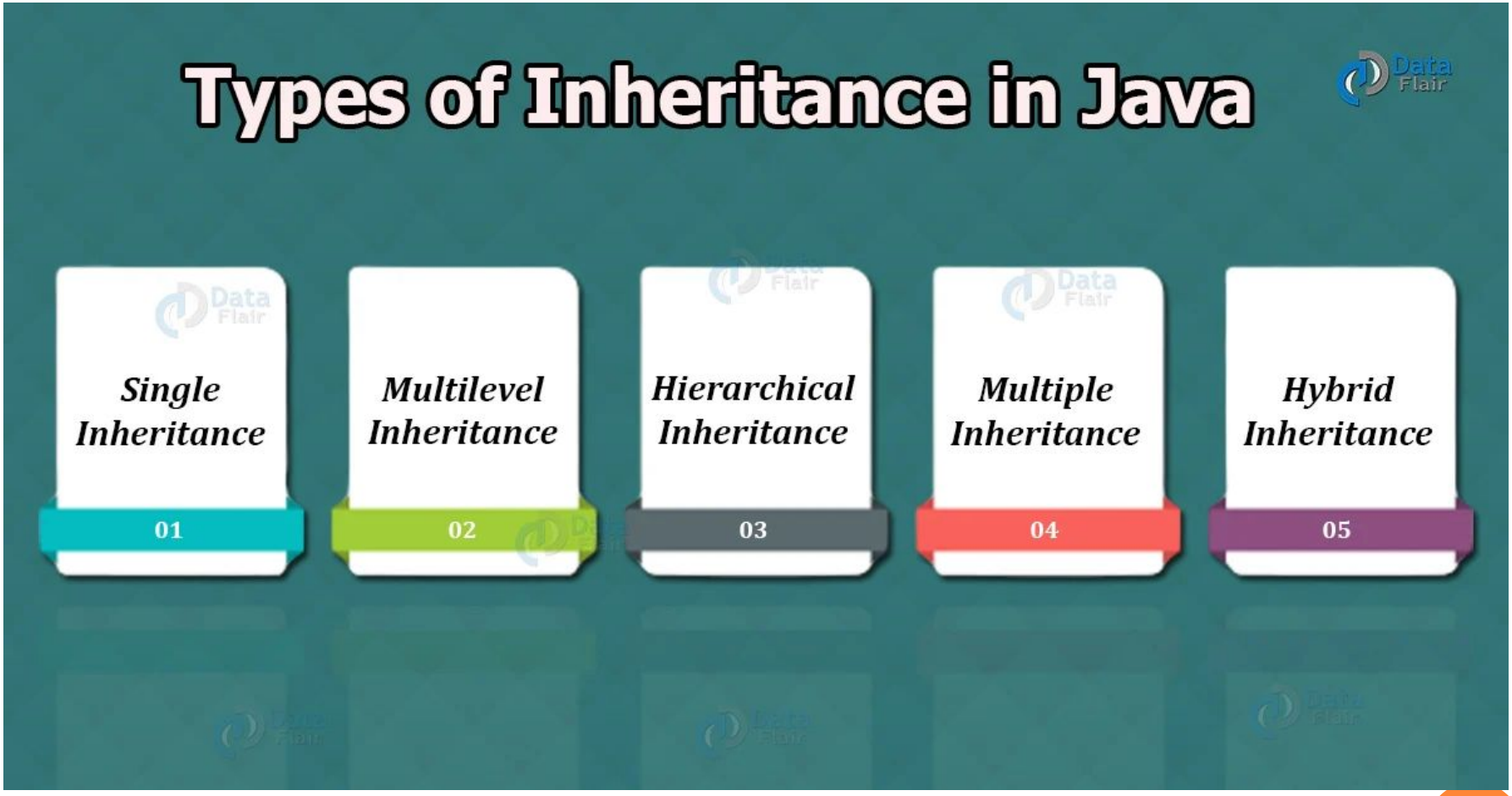
is-a relationship

In Java, inheritance is an **is-a relationship**. That is, we use inheritance only if there exists an is-a relationship between two classes.

For example,

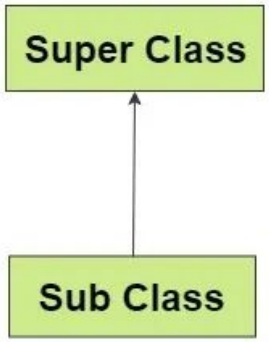
- **Car** is a **Vehicle**
- **Orange** is a **Fruit**
- **Surgeon** is a **Doctor**
- **Dog** is an **Animal**

TYPES OF INHERITANCE

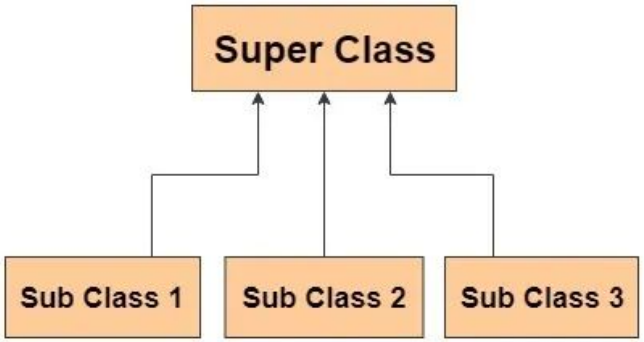


TYPES OF INHERITANCE

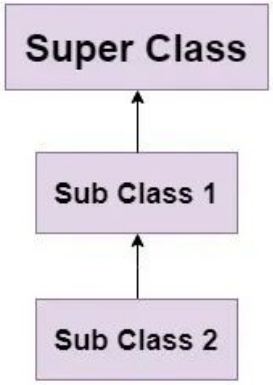
Single Inheritance



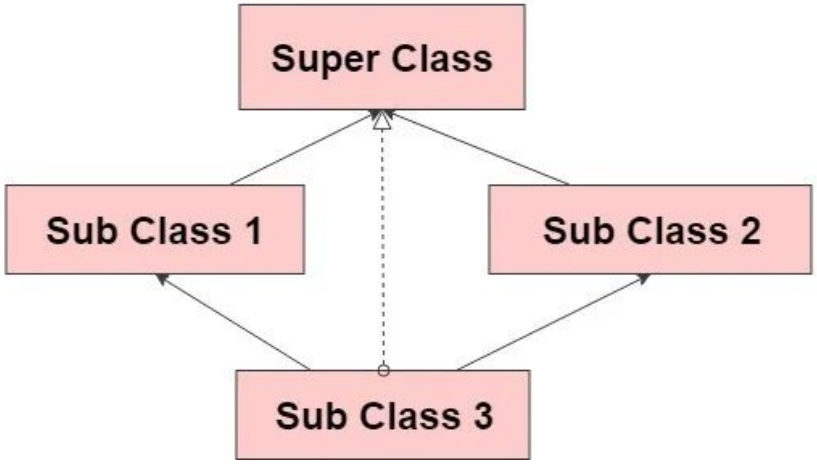
Hierarchial Inheritance



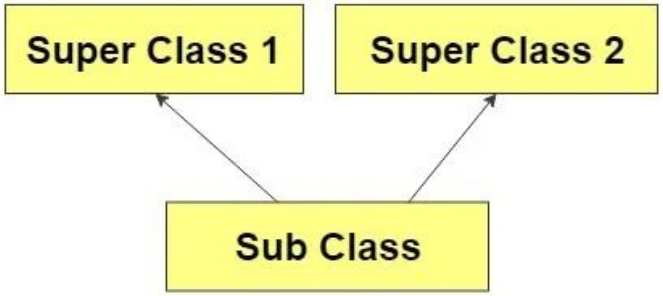
MultiLevel Inheritance



Hybrid Inheritance

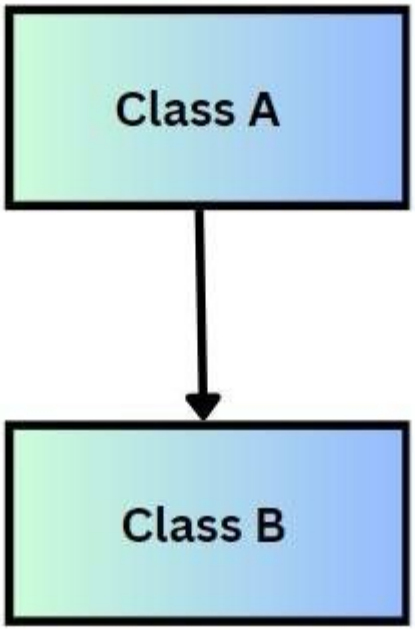


Multiple Inhertance



1. Single-level inheritance

- Single-level inheritance in Java is a fundamental concept where a subclass inherits from only one superclass.
- This means that the subclass can acquire the properties and methods of the superclass, enabling code reuse and the extension of existing functionality.

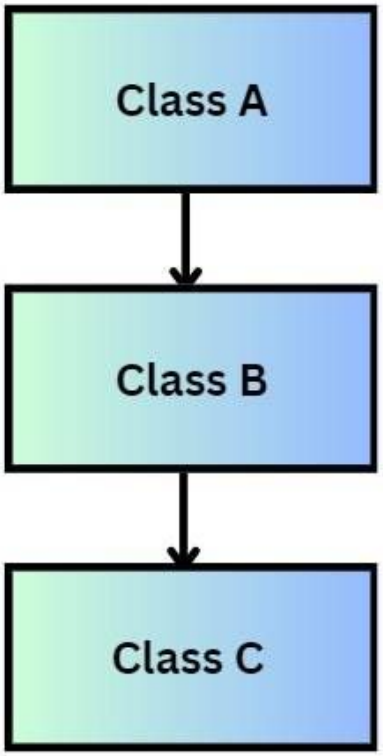


```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

// Child class
class Dog extends Animal {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
}

//Output:
```

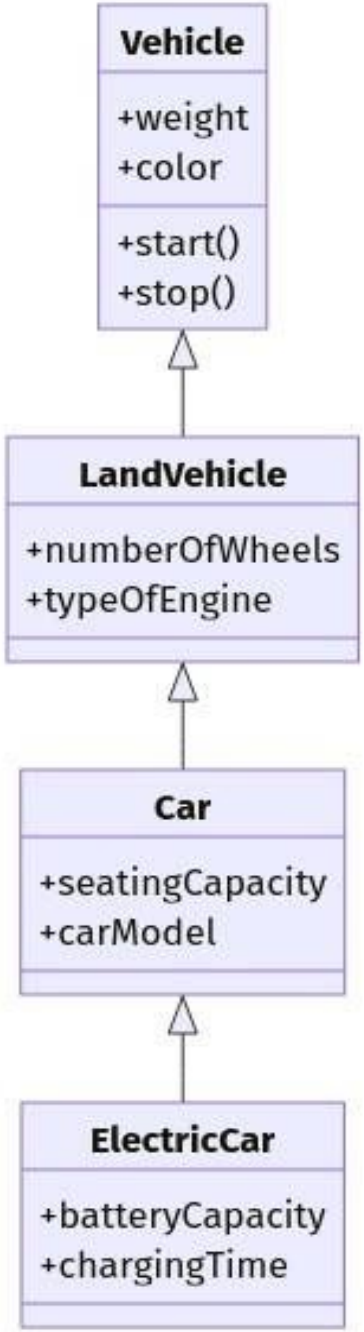
Animal makes a sound

2. Multi-level Inheritance

- Multilevel inheritance in Java is a feature that allows a class to inherit properties and behaviours from another class, which in turn inherits from another class, forming a "chain" of inheritance.
- This mechanism enables a class to inherit methods and fields from multiple ancestors but in a direct line, where each class in the chain inherits from one class directly above it.

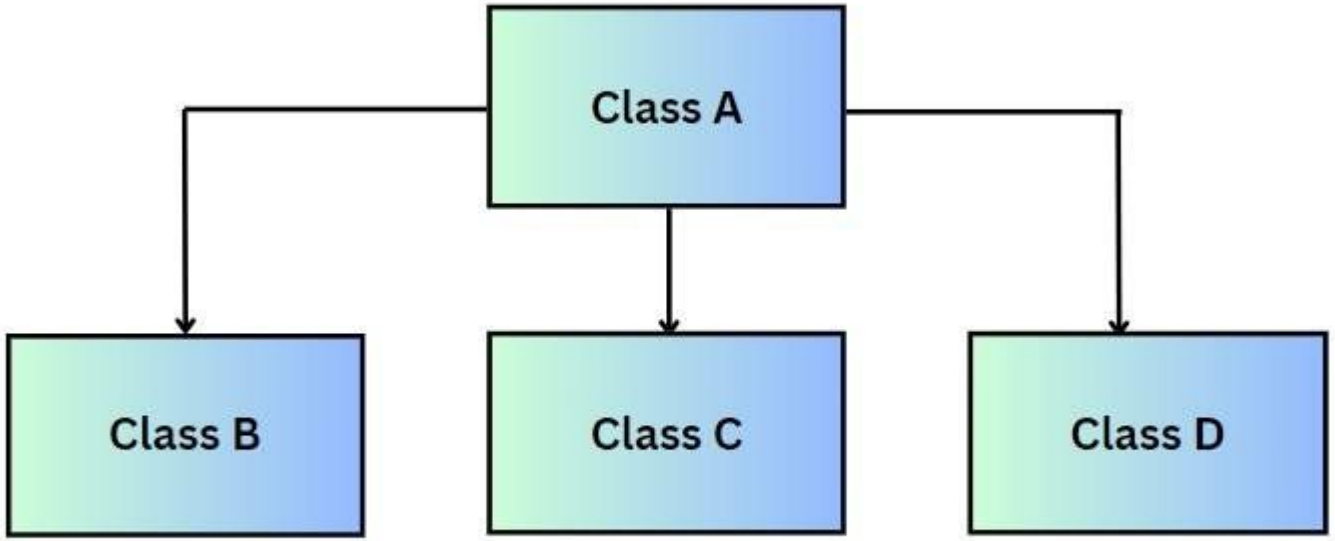


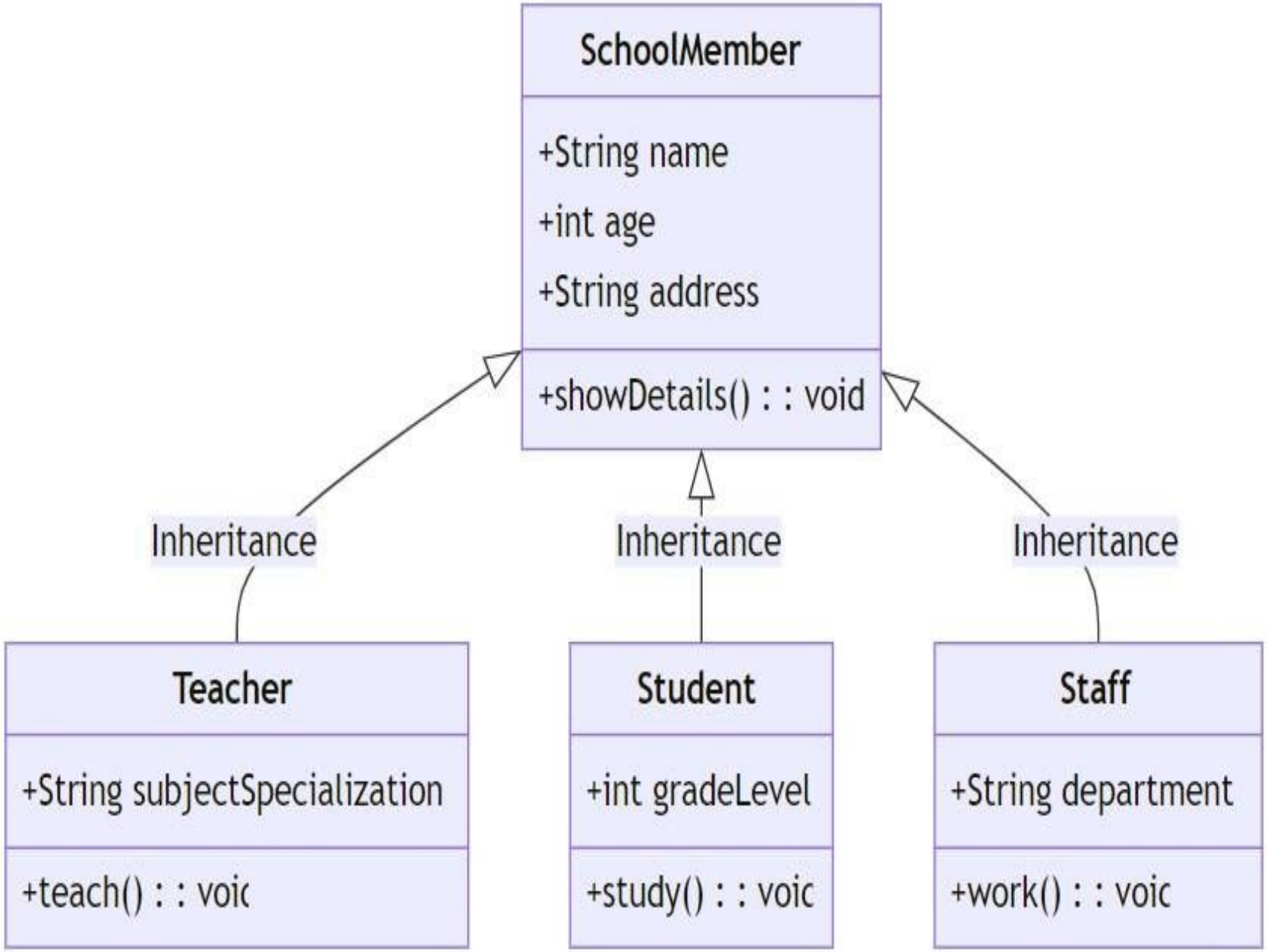
```
class Vehicle{
    //data members and methods
}

class LandVehicle extends Vehicle{
  //data members and methods

}
class Car extends LandVehicle{
    //data members and methods

}

class ElectricCar extends Car{
//data members and methods

}
```



3. Hierarchical Inheritance

- Hierarchical inheritance in Java occurs when one base class (superclass) is inherited by multiple subclasses.
- In this type of inheritance, all the features that are common in child classes are included in the base class.

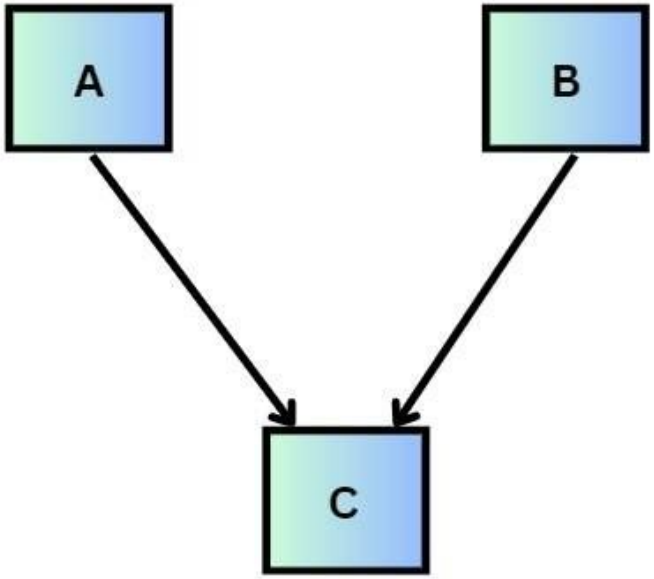



SchoolMember
+String name
+int age
+String address
+showDetails() : : void
Inheritance
Inheritance
Inheritance
Teacher
+String subjectSpecialization
+teach() : : voic
Student
+int gradeLevel
+study() : : voic
Staff
+String department
+work() : : voic

```
class SchoolMember{
        //data members and methods
}


class Teacher extends SchoolMember{
  //data members and methods

}


class Student extends SchoolMember{
      //data members and methods

}


class Staff extends SchoolMember{
  //data members and methods

}
```

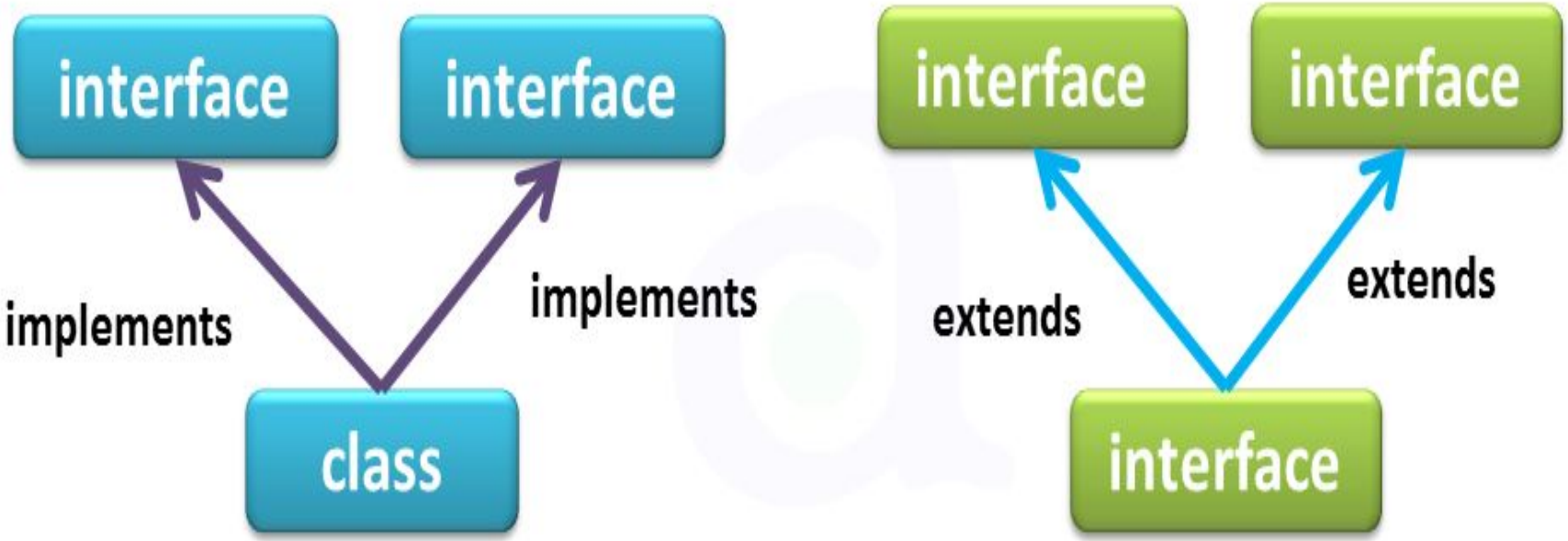
4. Multiple Inheritance

- Multiple inheritance refers to a feature in object-oriented programming where a class can inherit properties and methods from more than one parent class.
- This concept allows a subclass to inherit behaviours and attributes from multiple base.

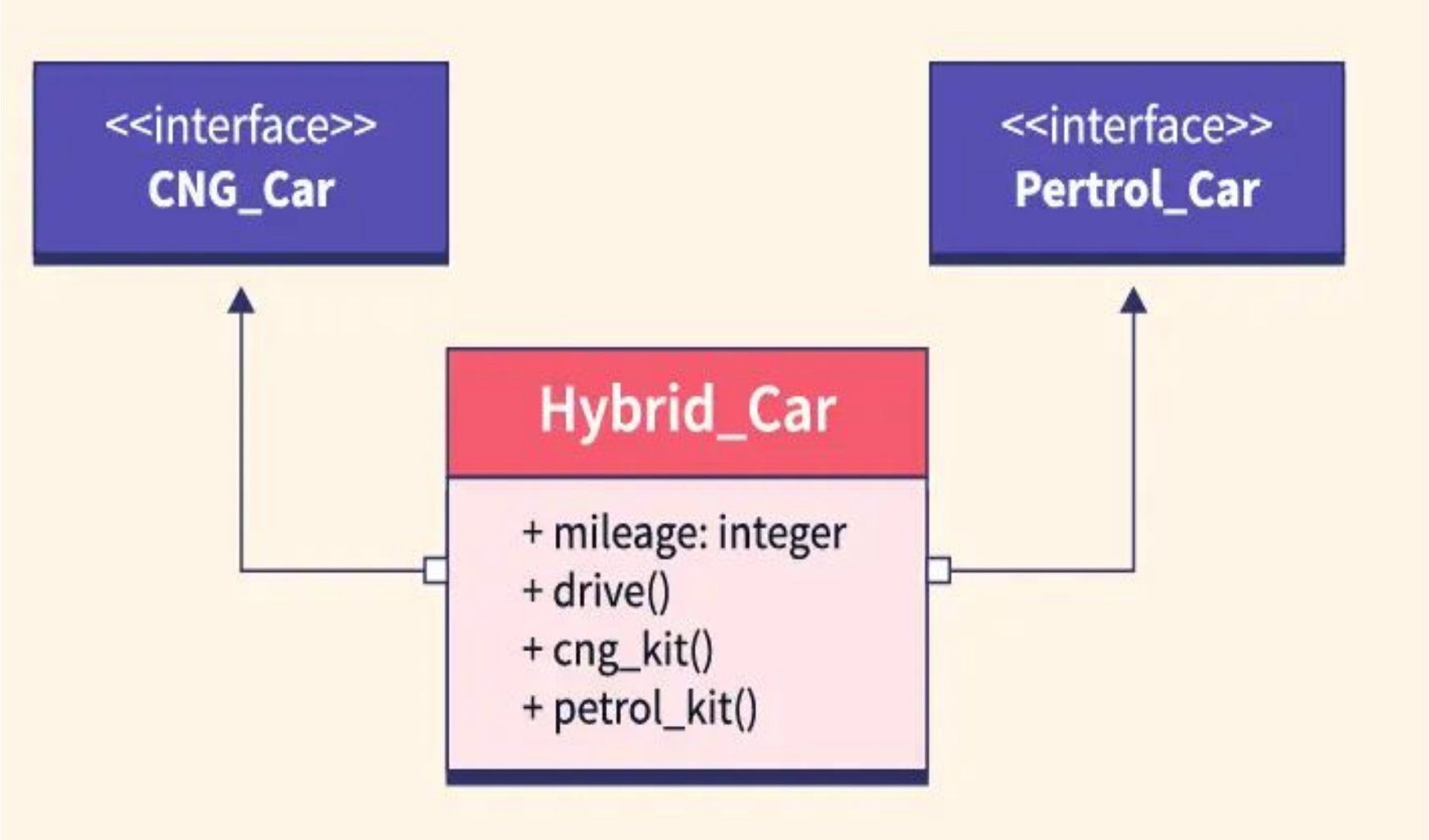


- However, it's important to note that **Java does not support multiple inheritance** for classes directly.

- Due to the complexity and potential for ambiguity it introduces (such as the "Diamond Problem", where a class inherits from two classes that have a common ancestor, leading to ambiguity in which ancestor's method or property to use).
- Instead of multiple inheritance for classes, Java provides a mechanism to achieve polymorphism and code reuse through **interfaces**.
- An interface in Java is a reference type, similar to a class, that can contain only constants, method signatures, default methods, static methods, and nested types.
- Interfaces cannot contain instance fields or constructor methods.
- A class in Java can implement multiple interfaces, allowing it to inherit abstract methods from multiple sources.



Multiple Inheritance in Java


<<interface>>
CNG_Car
<<interface>>
Pertrol_Car
Hybrid_Car
+ mileage: integer
+ drive()
+ cng_kit()
+ petrol_kit()

```java
// Define Interface 1
interface InterfaceA {
    void methodA();
}

// Define Interface 2
interface InterfaceB {
    void methodB();
}

// Class implementing multiple interfaces
class MyClass implements InterfaceA, InterfaceB {
    @Override
    public void methodA() {
        System.out.println("Implementation of methodA from InterfaceA");
    }
    @Override
    public void methodB() {
        System.out.println("Implementation of methodB from InterfaceB");
    }
 }
}
```

```java
public class Main {
    public static void main(String[] args) {
        MyClass obj = new MyClass();
        obj.methodA();
        obj.methodB();
    }
}
```

Output

Implementation of methodA from InterfaceA

Implementation of methodB from InterfaceB

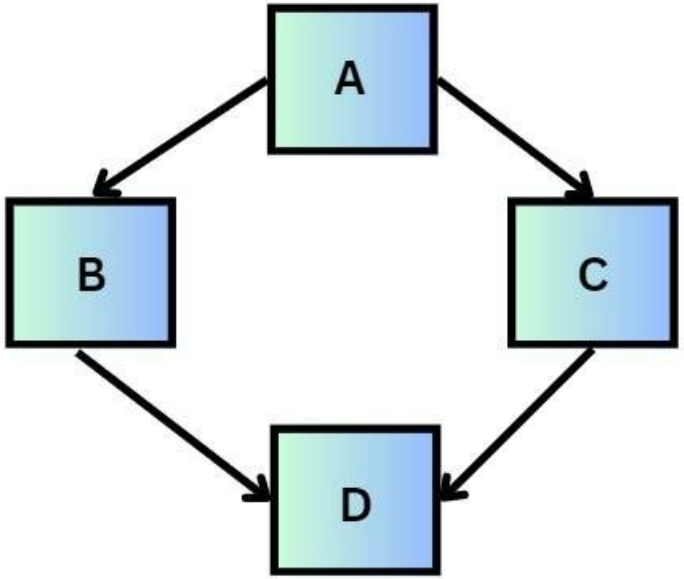
5. Hybrid Inheritance

- Hybrid inheritance is a combination of two or more types of inheritance, such as hierarchical, multiple, multilevel, or single inheritance.
- However, it's important to note that due to Java's restriction on extending multiple classes directly (to avoid complexity and ambiguity issues like the Diamond Problem), hybrid inheritance in Java is primarily achieved through a mix of class inheritance (using the extends keyword) and interface implementation (using the implements keyword).

In Java, hybrid inheritance can involve:

- **Single and Multilevel Inheritance through classes:** A class inherits from one superclass, and then another class inherits from that subclass, forming a linear inheritance chain.
- **Hierarchical Inheritance through classes:** Multiple classes inherit from a single parent class.
- **Multiple Inheritance through Interfaces:** A class implements multiple interfaces, inheriting the abstract methods from these interfaces.
- **Combining Class Inheritance and Interface Implementation:**

  A class extends another class and also implements one or more interfaces.



Syntax for Hybrid Inheritance (Conceptual Example):

The general idea is to have:

- **A base class**: from which multiple subclasses inherit (hierarchical inheritance).
- **One or more interfaces**: that define specific behaviors.
- **A final class**: that extends one of the subclasses from the hierarchy and implements one or more of the interfaces.

```
// 1. Base Class (Hierarchical Parent)

class Animal {

    void eat() {

        System.out.println("Animal is eating.");

    }

}
```

```java
// 2. Subclass 1 (Hierarchical Child)
class Mammal extends Animal {
    void breathe() {
        System.out.println("Mammal is breathing.");
    }
}

// 3. Interface for a specific behavior
interface Pet {
    void play();
}

// 4. Interface for another specific behavior
interface Domestic {
    void beFriendly();
}
```

```java
// 5. Final Class combining hierarchical inheritance and interface implementation
class Dog extends Mammal implements Pet, Domestic {
    void bark() {
        System.out.println("Dog is barking.");
    }

    @Override
    public void play() {
        System.out.println("Dog is playing fetch.");
    }

    @Override
    public void beFriendly() {
        System.out.println("Dog is wagging its tail.");
    }
}
```

```java
public class HybridInheritanceExample {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        myDog.eat();            // Inherited from Animal
        myDog.breathe();        // Inherited from Mammal
        myDog.bark();           // Specific to Dog
        myDog.play();           // Implemented from Pet interface
        myDog.beFriendly();     // Implemented from Domestic interface
    }
}
```

Programs
On
Inheritance

Program 1:

```
class Father {
  String familyName;
  String houseaddress;
  Father() {
    familyName = "Programmer";
    houseaddress = "Delhi";
  }
}
public class Son extends Father {
  String name;
  Son() {
    name = "Shraman";
  }
  void printdetails() {
    System.out.println("Hey my name is " + this.name + " " + this.familyName + " and I am from "
    + this.houseaddress);
  }
  public static void main(String[] args) {
    Son s1 = new Son();
    s1.printdetails();
  }
}
```

Program 2:

```
class GrandFather {
  GrandFather() {
    System.out.println("I am the grandfather!");
  }
}

class Father extends GrandFather {
  String familyName;
  String houseaddress;
  Father() {
    System.out.println("I am the father! I inherit from Grandfather");
  }
}
public class Son extends Father {

  Son() {
    System.out.println("I am the son and I inherit from my father.");
  }

  public static void main(String[] args) {
    Son s1 = new Son();
  }
}
```

Program 3:

```
class Father {
  String familyName;
  String houseaddress;
  Father() {
    familyName = "Programmer";
    houseaddress = "Delhi";
  }
}
class Son extends Father {
  Son() {
    System.out.println("I am the Son");
    System.out.println("My family name is " + this.familyName + " and I am from " + this.houseaddress);

  }

}
class Daughter extends Father {
  Daughter() {
    System.out.println("I am the Daughter");
    System.out.println("My family name is " + this.familyName + " and I am from " + this.houseaddress);
  }
}
class Main {
  public static void main(String[] args) {
    Son s = new Son();
    Daughter d = new Daughter();
  }
}
```

Program 4:

```
interface first {
  public void firstmethod();
}
interface second {
  public void secondmethod();
}
interface third extends first,second {
  public void mainmethod();
}

public class MultInheritance implements third {
  public void mainmethod() {
    System.out.println("This is the main interface method");
  }
  public void firstmethod() {
    System.out.println("Hey I am the first method defined the first interface");
  }
  public void secondmethod() {
    System.out.println("Hey I am the second method defined it the second interface");
  }
  public static void main(String[] args) {
    MultInheritance ob = new MultInheritance();
    ob.mainmethod();
    ob.firstmethod();
    ob.secondmethod();
  }
}
```

Program 5:

```
class A {
  A() {
    System.out.println("I am in class A! I have two children B and C");
  }
}
class B extends A {
  B() {
    System.out.println("I am in class B! I have 1 child D");
  }

}
class C extends A {
  C() {
    System.out.println("I am in class C. I am the child class of A");
  }
}
class D extends B {
  D() {
    System.out.println("I am in D class which is the extension of B class");
  }
}
class Main {
  public static void main(String[] args) {
    D d = new D(); //This calls constructors of parent class A and B
    B b = new B(); //This calls constructors parent class A and B
    C c = new C(); //This calls the constructors class A and C
  }
}
```

CONSTRUCTORS IN SUBCLASSES

- In Java, constructors are not inherited by subclasses.

- However, when an object of a subclass is created, the constructor of its superclass is always invoked, either explicitly or implicitly.

- **Note: In Java, constructor of the base class with no argument gets automatically called in the derived class constructor.**

Example:

```
class Base {

    Base()
    {
        System.out.println("Base Class Constructor Called ");
    }
}

class Derived extends Base {

    Derived()
    {
        System.out.println("Derived Class Constructor Called ");
    }
}
```

```
class Demo {

    public static void main(String[] args)
    {

        Derived d = new Derived();

    }
}
```

Output

```
Base Class Constructor Called
Derived Class Constructor Called
```

- If we want to call a parameterized constructor of the base class, then we can call it using **super()**.

- The point to note is base class constructor call **must be the first line** in the derived class constructor.

```
class Base {
    int x;
    Base(int x1) {
        x = x1;
    }
}
```

```
class Derived extends Base {
    int y;
    Derived(int x1, int y1)
    {
        super(x1);
        y = y1;
    }
    void Display()
    {
       System.out.println("x = " + x + ", y = " + y);
    }
}
public class GFG {

    public static void main(String[] args)
    {
            Derived d = new Derived(10, 20);
            d.Display();
    }
}
```

Output

```
x = 10, y = 20
```

- **Constructor Chaining:**
- When a subclass object is instantiated, a chain of constructor calls occurs, starting from the constructor of Object (the ultimate superclass of all Java classes) and proceeding down the inheritance hierarchy to the actual subclass constructor.
- This ensures that all inherited fields are properly initialized.

Java Constructor Chaining

TheCodingShala.co

```
Student(){
        this(5);
}
```

```
Student(int id){
        this(id, "hi")
}
```

```
Student(int id, String
msg){
//print
}
```

Important Points Regarding Inheritance in Java

1. Default Superclass

Every class in java is a subclass of one and only one class in Java. There is one exception to this and that is the Object class. The object class does not have any superclass.

2. No Multiple Inheritance in Java

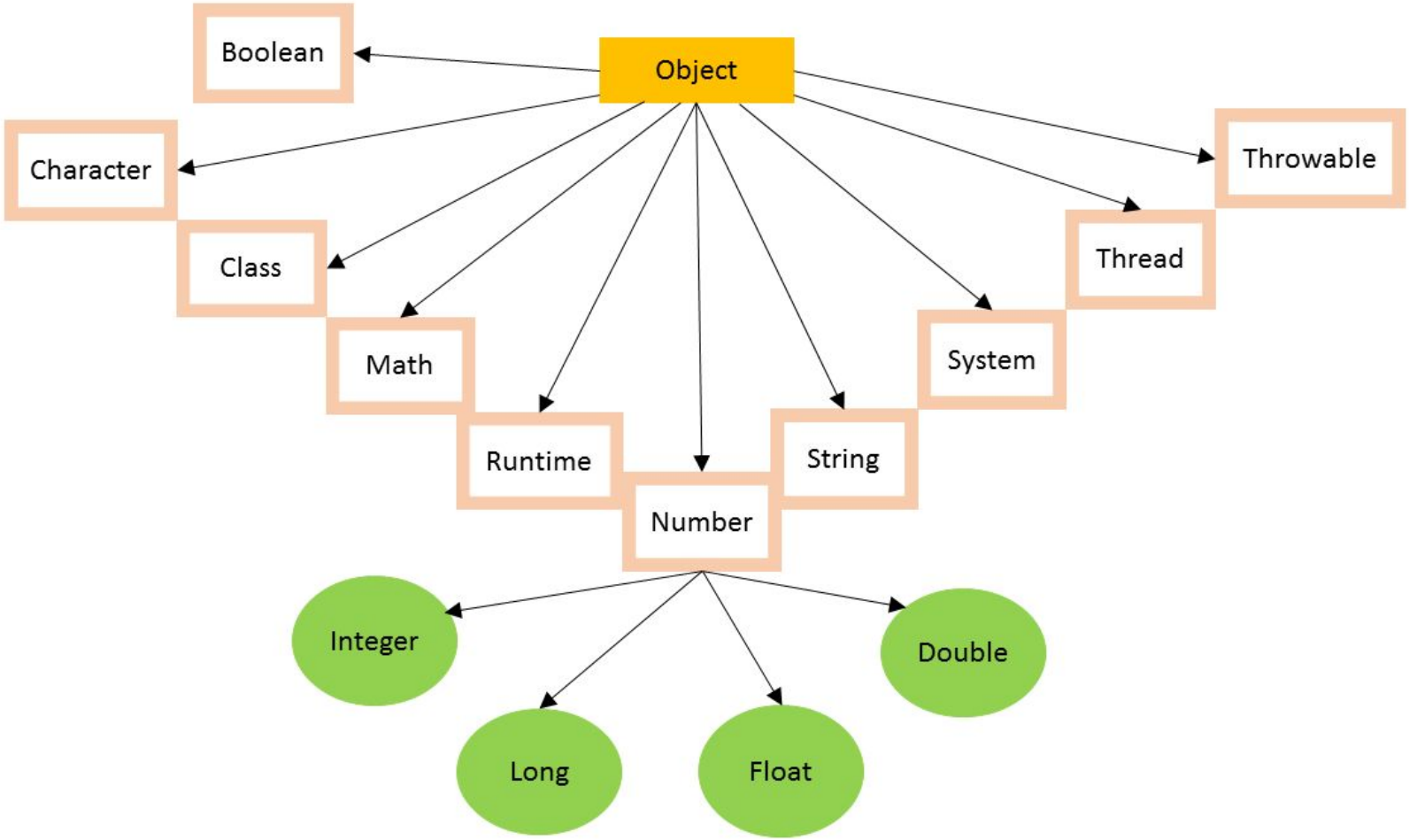
3. Constructor Inheritance in Java

Constructors are not inherited into the child classes. However they are inherited with the help of super() keyword.

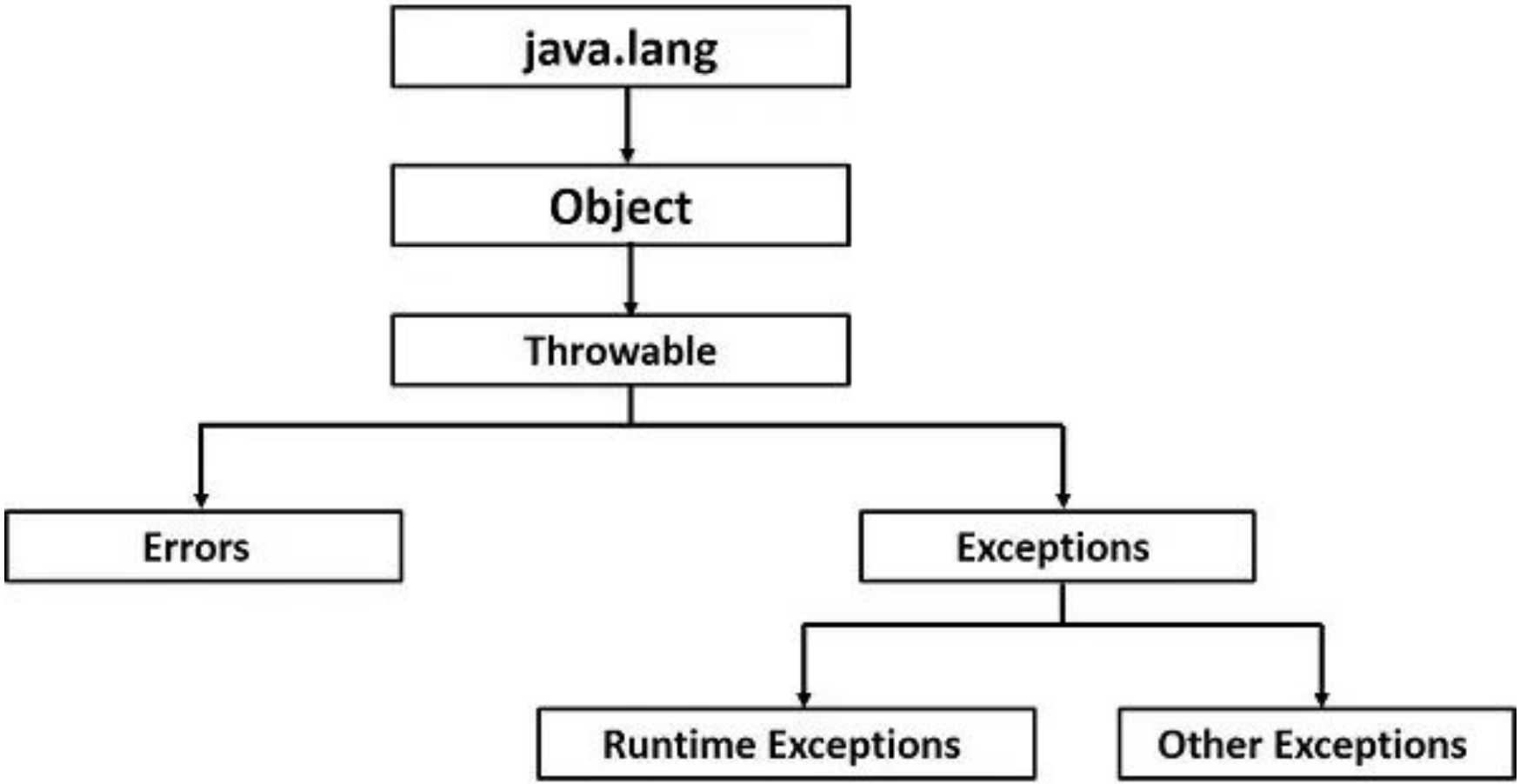
4. Inheriting Private Members in Java

The subclass cannot access the private members of the parent class.

Object Class in Java



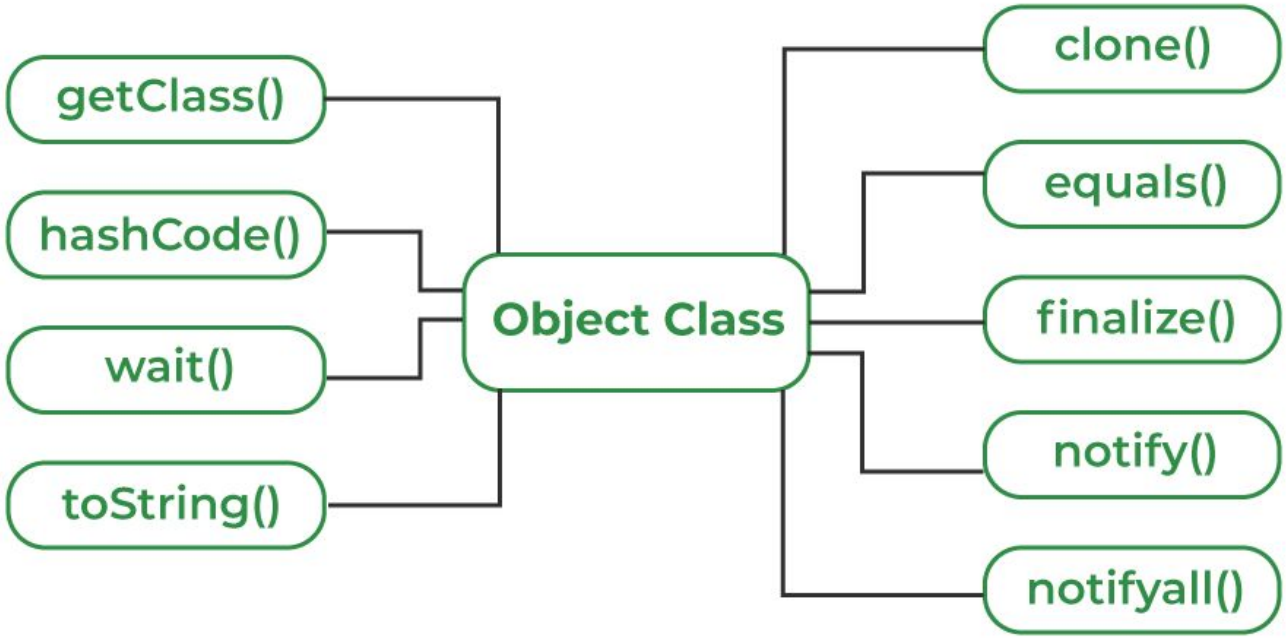
https://docs.oracle.com/javase/8/docs/api/java/lang/package-tree.html


java.lang
Object
Throwable
Errors
Exceptions
Runtime Exceptions
Other Exceptions

- In Java, the class hierarchy is a tree-like structure where all classes ultimately descend from a single root class called Object.
- This Object class is located in the java.lang package and serves as the universal superclass for every other class in Java.
- **How the object class hierarchy works in Java:**
- **Root of the Hierarchy:** The java.lang.Object class sits at the very top of the class hierarchy. All other classes, whether built-in or user-defined, are direct or indirect subclasses of Object.
- **Implicit Inheritance:** If a class does not explicitly extend another class using the extends keyword, it implicitly extends Object. For example, a simple class declaration like class MyClass {} is equivalent to class MyClass extends Object {}.

- **Inherited Methods:**

  Because Object is the superclass of all classes, its methods are inherited by every Java class. These methods include equals(), hashCode(), toString(), getClass(), notify(), notifyAll(), and wait(). Subclasses can override these methods to provide specific implementations for their own types.



- **Specialization through Inheritance:** As you move down the class hierarchy, classes become more specialized. For example, Object is the most general class, while classes like String, ArrayList, or custom classes you create are more specific, inheriting general behaviors from Object and adding their own unique functionalities.

In essence, the Object class forms the foundation of the entire Java type system, ensuring that all objects share a common set of fundamental behaviors and providing a unified structure for inheritance.

- **Object Class Declaration**

```
public class Object
```

- **Class constructors**

Object()

This is the Single Constructor.

-

- **Class methods**

Sr.No.	Method & Description
1	**protected Object clone()** This method creates and returns a copy of this object.
2	**boolean equals(Object obj)** This method indicates whether some other object is "equal to" this one.
3	**protected void finalize()** This method is called by the garbage collector on an object when garbage collection determines that there are no more references to the object.
4	**Class<?> getClass()** This method returns the runtime class of this Object.
5	**int hashCode()** This method returns a hash code value for the object.
6	**void notify()** This method wakes up a single thread that is waiting on this object's monitor.

7	**void notifyAll()** This method wakes up all threads that are waiting on this object's monitor.
8	**String toString()** This method returns a string representation of the object.
9	**void wait()** This method causes the current thread to wait until another thread invokes the notify() method or the notifyAll() method for this object.
10	**void wait(long timeout)** This method causes the current thread to wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed.
11	**void wait(long timeout, int nanos)** This method causes the current thread to wait until another thread invokes the notify() method or the notifyAll() method for this object, or some other thread interrupts the current thread, or a certain amount of real time has elapsed.

Polymorphism:

- Abstract classes and methods,
- final methods and classes,
- dynamic binding,
- polymorphism examples and
- Interfaces.

Polymorphism

- **Polymorphism in Java** is one of the core concepts in object-oriented programming (OOP) that allows objects to behave differently based on their specific class type.

- The word **polymorphism** means having **many forms**, and it comes from the Greek words **poly (many)** and **morph (forms)**.

- This means one entity can take many forms. In Java, polymorphism allows the same method or object to behave differently based on the context.

Key features of polymorphism:

- **Multiple Behaviors**: The same method can behave differently depending on the object that calls this method.
- **Method Overriding**: A child class can redefine a method of its parent class.
- **Method Overloading**: We can define multiple methods with the same name but different parameters.
- **Runtime Decision:** At runtime, Java determines which method to call depending on the object's actual class.

Example :

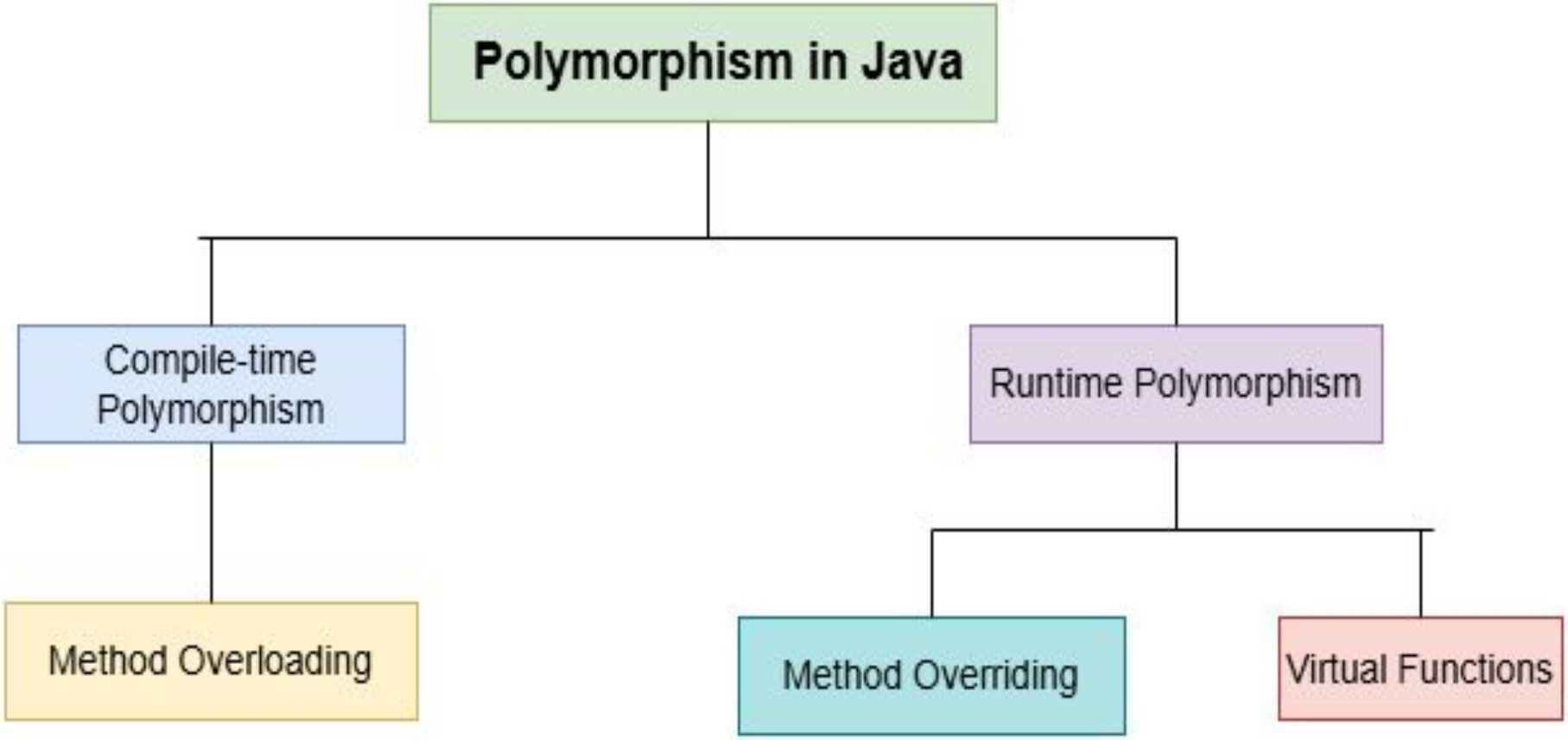
```
// Base class Person
class Person {

    // Method that displays the role of a person
    void role() {
        System.out.println("I am a person.");
    }
}

class Father extends Person {

    // Overridden method to show the role of a father
    @Override
    void role() {
        System.out.println("I am a father.");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        // Creating a reference of type Person
        // but initializing it with Father class object
        Person p = new Father();

        // Calling the role method. It calls the
        // overridden version in Father class
        p.role();
    }
}
```

Output

```
I am a father.
```

Types of Polymorphism in Java



In Java Polymorphism is mainly divided into two types:

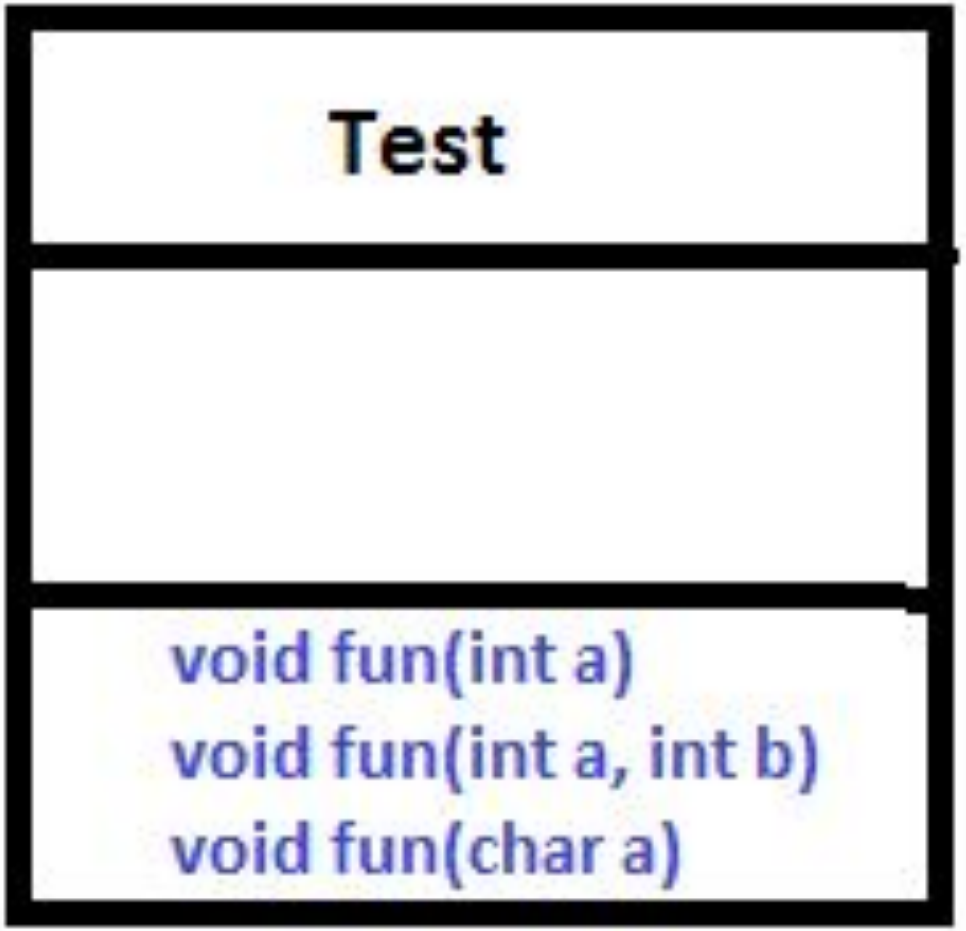
1. **Compile-Time Polymorphism (Static) / Early Binding**
2. **Runtime Polymorphism (Dynamic) / Late Binding**

1. Compile-Time Polymorphism

- Compile-Time Polymorphism in Java is also known as **static polymorphism** and also known as method overloading.
- This happens when **multiple methods** in the **same class** have the **same name but different parameters**.
- ***Note**: But Java doesn't support the Operator Overloading.*

1) Method Overloading

- Method Overloading allows us to define multiple methods with the same name but different parameters within a class.
- The different ways of method overloading in Java are mentioned below:
  - **Changing the Number of Parameters**
  - **Changing Data Types of the Arguments**
  - **Changing the Order of the Parameters of Methods**
- Method overloading in Java is also known as **Compile-time Polymorphism**, **Static Polymorphism, or Early binding**, because the decision about which method to call is made at compile time.



Overloading

1.Method overloading by Changing the Number of Parameters

```
import java.io.*;
class Product {
    // Method 1 // Multiplying two integer values
   public int multiply(int a, int b)
   {
      int prod = a * b;
      return prod;
   }
 // Method 2  // Multiplying three integer values
   public int multiply(int a, int b, int c)
   {
      int prod = a * b * c;
      return prod;
   }
```

```
public static void main(String[] args)
  {
    // Creating object of above class inside main()
    // method
    Product ob = new Product();

    // Calling method to Multiply 2 numbers
    int prod1 = ob.multiply(1, 2);

     System.out.println("Product of the two integer value: " + prod1);

    // Calling method to multiply 3 numbers
    int prod2 = ob.multiply(1, 2, 3);

    System.out.println("Product of the three integer value: " + prod2);
  }
}
```

2. Method overloading by Changing Data Types of the Arguments

```
class Product {
    // Multiplying three integer values
    public int Prod(int a, int b, int c)
    {
        int prod1 = a * b * c;
        return prod1;
    }
```

```
// Multiplying three double values.
    public double Prod(double a, double b, double c)
    {
        double prod2 = a * b * c;
        return prod2;
    }
}
```

```
class Geeks {
    public static void main(String[] args)
    {
        Product obj = new Product();

        int prod1 = obj.Prod(1, 2, 3);
    System.out.println("Product of the three integer value: " + prod1);

        double prod2 = obj.Prod(1.0, 2.0, 3.0);
        System.out.println("Product of the three double value: " + prod2);
    }
}
```

Output

```
Product of the three integer value: 6
Product of the three double value: 6.0
```

3. Method Overloading By Changing the Order of the Parameters of Methods

```
class Student {

    // Method 1
    public void StudentId(String name, int roll_no)
    {
        System.out.println("Name: " + name + " "
                           + "Roll-No: " + roll_no);
    }

    // Method 2
    public void StudentId(int roll_no, String name)
    {
        // Again printing name and id of person
        System.out.println("Roll-No: " + roll_no + " "
                           + "Name: " + name);
    }
}
```

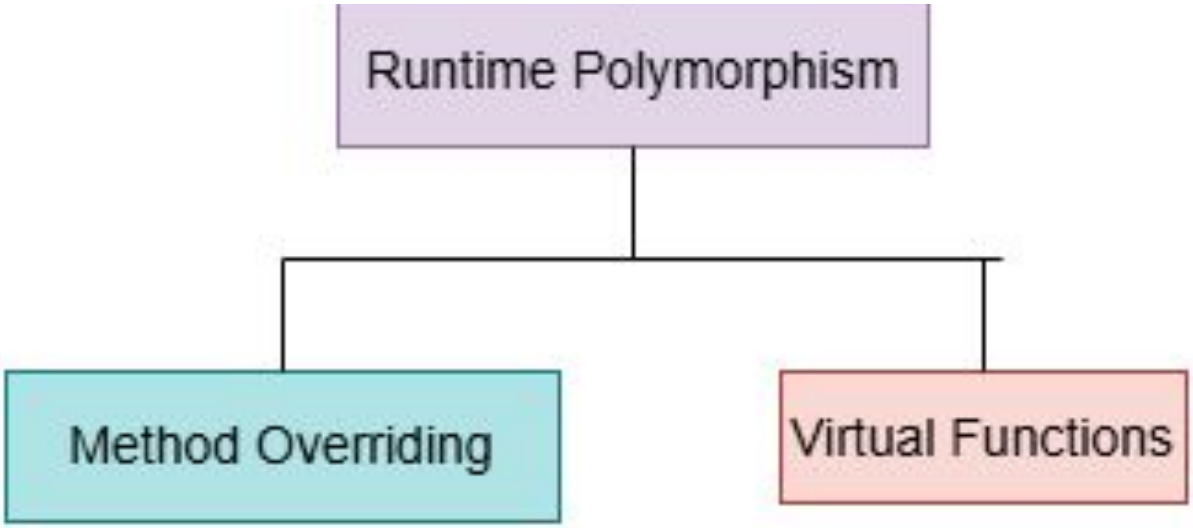
```
class Geeks {

    // Main function
    public static void main(String[] args)
    {
        // Creating object of above class
        Student obj = new Student();

        // Passing name and id
        // Note: Reversing order
        obj.StudentId("Sweta", 1);
        obj.StudentId(2, "Gudly");
    }
}
```

Output

```
Name: Sweta Roll-No: 1
Roll-No: 2 Name: Gudly
```

2 Runtime Polymorphism OR Dynamic / Late Binding

- Runtime Polymorphism in Java known as Dynamic Method Dispatch.
- It is a process in which a function call to the overridden method is resolved at Runtime.
- This type of polymorphism is achieved by Method Overriding.



1) Method Overriding

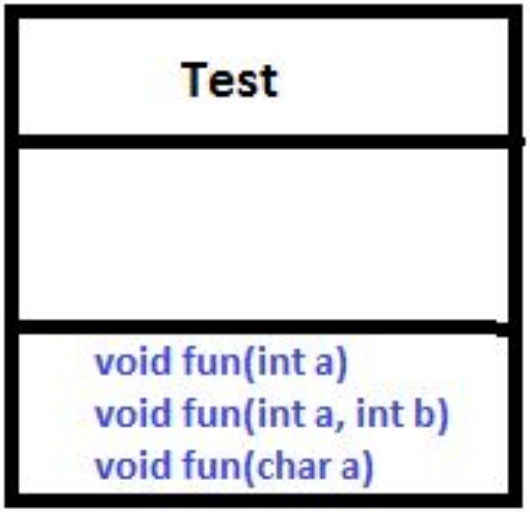
- Method overriding in Java means when a subclass provides a specific implementation of a method that is already defined in its superclass.
- The method in the subclass must have the same name, return type, and parameters as the method in the superclass.
- Method overriding allows a subclass to modify or extend the behavior of an existing method in the parent class.
- This enables dynamic method dispatch, where the method that gets executed is determined at runtime based on the object's actual type.

```java
class Animal {
   public void move() {
      System.out.println("Animals can move");
   }
}
class Dog extends Animal {
   public void move() {
      System.out.println("Dogs can walk and run");
   }
}
public class TestDog {
   public static void main(String args[]) {
      Animal a = new Animal(); // Animal reference and object
      Animal b = new Dog(); // Animal reference but Dog object
      a.move(); // runs the method in Animal class
      b.move(); // runs the method in Dog class
   }
}
```

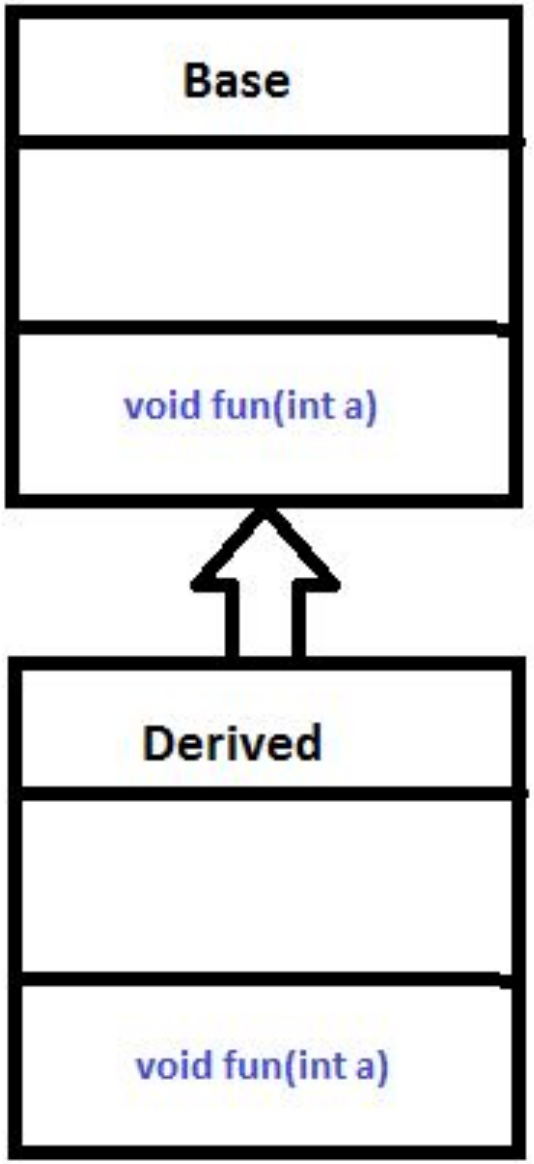
```
Animals can move
Dogs can walk and run
```

```
class Parent {

    void Print() {
      System.out.println("parent class");
   }
}
class subclass1 extends Parent {

    void Print() {
    System.out.println("subclass1");
   }
}
class subclass2 extends Parent {

   void Print() {
      System.out.println("subclass2");
   }
}
```

```
class Geeks {

    public static void main(String[] args) {

        Parent a;

        a = new subclass1();
        a.Print();

        a = new subclass2();
        a.Print();
    }
}
```



Overloading



Overriding

Virtual Function In Java

- The behavior of a virtual function can be overridden with the inheriting class function with the same name. It is basically defined in the base class and overridden in the inherited class.
- Virtual function in Java is expected to be defined in the derived class. We can call the virtual function by referring to the object of the derived class using the reference or pointer of the base class.

- Every non-static method in Java is by default a virtual method. Java does not have a virtual keyword like C++, but we can define them and use them for concepts like run-time polymorphism.

Virtual Function Example

```
class Vehicle{
void make(){
System.out.println("heavy duty");
}
}
public class Trucks extends Vehicle{
void make(){
System.out.println("Transport vehicle for heavy duty");
}
public static void main(String args[]){
Vehicle ob1 = new Trucks();
ob1.make();
}
}
```

Output: Transport vehicle for heavy duty

Advantages of Polymorphism

- Encourages code reuse.
- Simplifies maintenance.
- Enables dynamic method dispatch.
- Helps in writing generic code that works with many types.

Disadvantages of Polymorphism

- It can make more difficult to understand the behavior of an object.
- This may cause performance issues, as polymorphic behavior may require additional computations at runtime.

Abstract classes and methods

- In Java, abstract class is declared with the abstract keyword.
- It may have both abstract and non-abstract methods(methods with bodies).
- An abstract is a Java modifier applicable for classes and methods in Java but *not for Variables*.
- Java abstract class is a class that can not be instantiated by itself, it needs to be subclassed by another class to use its properties.
- An abstract class is declared using the "**abstract**" keyword in its class definition.

Illustration of Abstract class

```
abstract class Shape
{
    int color;
    // An abstract function
    abstract void draw();
}
```

In Java, the following some *important observations* about abstract classes are as follows:

1. An instance of an abstract class can not be created.
2. Constructors are allowed.
3. We can have an abstract class without any abstract method.
4. There can be a **final method** in abstract class but any abstract method in class(abstract class) can not be declared as final or in simpler terms final method can not be abstract itself as it will yield an error: "Illegal combination of modifiers: abstract and final"

5. We can define static methods in an abstract class

6. We can use the **abstract keyword** for declaring ***top-level classes (Outer class) as well as inner classes*** as abstract

7. If a **class** contains at least **one abstract method** then compulsory should declare a class as abstract

8. If the **Child class** is unable to provide implementation to all abstract methods of the **Parent class** then we should declare that **Child class as abstract** so that the next level Child class should provide implementation to the remaining abstract method.

1. Example of Abstract Class that has Abstract method

```java
// Abstract class
abstract class Sunstar {
    abstract void printInfo();
}

// Abstraction performed using extends
class Employee extends Sunstar {
    void printInfo()
    {
        String name = "avinash";
        int age = 21;
        float salary = 222.2F;

        System.out.println(name);
        System.out.println(age);
        System.out.println(salary);
    }
}
```

```java
// Base class
class Base {
    public static void main(String args[])
    {
        Sunstar s = new Employee();
        s.printInfo();
    }
}
```

Output

```
avinash
21
222.2
```

2. Abstract Class having constructor, data member, and methods

```
abstract class Subject {
    Subject() {
      System.out.println("Learning Subject");
    }

      abstract void syllabus();

      void Learn(){
          System.out.println("Preparing Right Now!");
    }
}

class IT extends Subject {
  void syllabus(){
    System.out.println("C , Java , C++");
  }
}
```

```
class GFG {
    public static void main(String[] args) {
        Subject x=new IT();

          x.syllabus();
          x.Learn();
    }
}
```

Output

```
Learning Subject
C , Java , C++
Preparing Right Now!
```

FINAL METHODS AND CLASSES

- There are **48 reserved keywords** in Java and the **final** keyword is one of them.
- The final keyword in Java is a **non-access modifier** used to **prevent modification.**
- It can be applied to

  1) variables (value cannot change),

  2) methods (cannot be overridden) and

  3) classes (cannot be extended).

Final Variable → To Create constant variable

Final Methods → Prevent Method Overriding

Final Classes → Prevent Inheritance

Characteristics of final keyword in Java

- **Final variables:** Declaring a variable as final prevents its value from being changed after initialization, useful for constants.
- **Final methods**: When a method is declared as final, it cannot be overridden by a subclass. This is useful for methods that are part of a class's public API and should not be modified by subclasses.
- **Final classes:** When a class is declared as final, it cannot be extended by a subclass. This is useful for classes that are intended to be used as is and should not be modified or extended.

- **Initialization:** Final variables must be initialized either at the time of declaration or in the constructor of the class. This ensures that the value of the variable is set and cannot be changed.
- **Performance:** The use of a final can sometimes improve performance, as the compiler can optimize the code more effectively when it knows that a variable or method cannot be changed.
- **Security:** The final can help improve security by preventing malicious code from modifying sensitive data or behavior.

Java Final Variable

```java
public class Geeks {
    public static void main(String[] args) {

        // Define a constant variable PI
        final double PI = 3.14159;

        // Print the value of PI
        System.out.println("Value of PI: " + PI);
    }
}
```

Output

```
Value of PI: 3.14159
```

Different Methods of Using Final Variable

1. final Variable

Example:

```
// Final variable
final int THRESHOLD = 5;
```

2. Blank final Variable

Example:

```
// Blank final variable
final int THRESHOLD;
```

3. Static final Variable

Example:

```
// Final static variable PI
static final double PI = 3.141592653589793;
```

4. Static Blank final Variable

Example:

```
// Blank final static  variable
static final double PI;
```

Initializing a final Variable

1. **At declaration:** the most common approach.
2. **In a constructor or instance initializer block:** required for blank final instance variables. If multiple constructors exist, all must initialize the variable.
3. **In a static block:** used for blank final static variables.

```java
class Geeks {

    // a final variable direct initialize
    final int THRESHOLD = 5;

    // a blank final variable
    final int CAPACITY;

    // another blank final variable
    final int  MINIMUM;

    // a final static variable PI direct initialize
    static final double PI = 3.141592653589793;

    // a  blank final static  variable
    static final double EULERCONSTANT;
```

```java
    // instance initializer block for initializing CAPACITY
    {
        CAPACITY = 25;
    }

    // static initializer block for initializing EULERCONSTANT
    static{
        EULERCONSTANT = 2.3;
    }

    // constructor for initializing MINIMUM
    public Geeks()
    {
        MINIMUM = -1;
    }

}
```

Final Method in Java

- We can declare Java methods as Final Method by adding the **Final keyword before the method name.**
- The Method with Final Keyword **cannot be overridden in the subclasses.**

```
class SuperClass {
    public static final void foo() {
        System.out.println("at BaseClass no-args foo");
    }
}

class SubClass extends SuperClass {
    public static void foo() {
        System.out.println("at SubClass no-args foo");
    }
}
```

Illustration: Final keyword with a method

```
class A
{
    final void m1()
    {
        System.out.println("This is a final method.");
    }
}
class B extends A
{
    void m1()
    {
        // Compile-error! We can not override
        System.out.println("Illegal!");
    }
}
```

Java Final classes

- When a class is declared with the final keyword in Java, it is called a final class.
- A final class cannot be extended(inherited).

Uses of a final class:

1. Usage 1: One is definitely to prevent inheritance

1. Usage 2: The other use of final with classes is to create an immutable class like the predefined String class.

Usage 1: One is definitely to prevent inheritance, as final classes cannot be extended.

```
final class A
{
    // methods and fields
}
// The following class is illegal
class B extends A
{
    // COMPILE-ERROR! Can't subclass A
}
```

Advantages of final Keyword in Java

- Ensuring immutability
- Improving performance
- Enhancing security
- Promoting code reuse
- Making code easier to understand

DYNAMIC BINDING

- Dynamic binding, also known as late binding or runtime polymorphism, in Java refers to the mechanism where the method implementation to be executed is determined at runtime, based on the actual type of the object, rather than at compile time based on the reference type.

- Parent Class Reference, Child Class Object

```
Animal myAnimal = new Dog(); // Animal is superclass, Dog is subclass
myAnimal.makeSound(); // Which makeSound() is called?
```

Example of dynamic binding

```
class Animal{
 void eat(){System.out.println("animal is eating...");}
}

class Dog extends Animal{
 void eat(){System.out.println("dog is eating...");}

 public static void main(String args[]){
  Animal a=new Dog();
  a.eat();
 }
}
```

Static Binding	Dynamic Binding
It takes place at compile time for which is referred to as early binding	It takes place at runtime so do it is referred to as late binding.
It uses overloading more precisely operator overloading method	It uses overriding methods.
It takes place using normal functions	It takes place using virtual functions
Static or const or private functions use real objects in static binding	Real objects use dynamic binding.

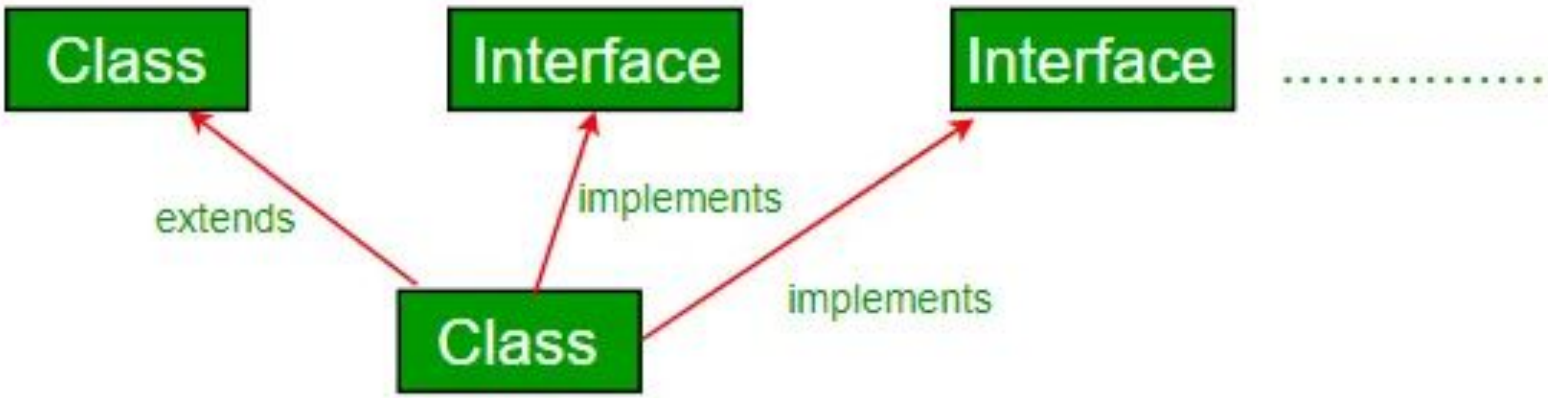
POLYMORPHISM EXAMPLES

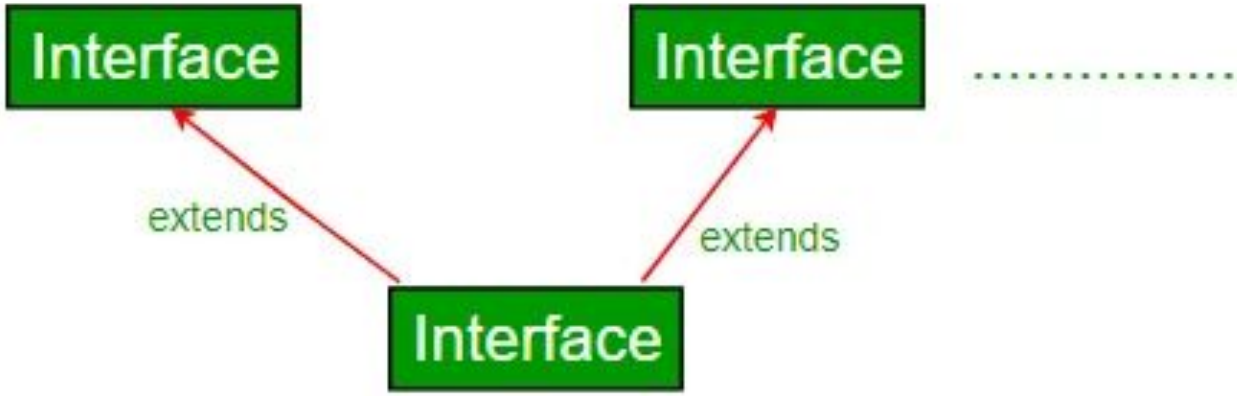
create a superclass: `Polygon` and two subclasses: `Square` and `Circle`. Define the `render()` method in all classes.

```
class Polygon {

  // method to render a shape
  public void render() {
    System.out.println("Rendering Polygon...");
  }
}

class Square extends Polygon {

  // renders Square
  public void render() {
    System.out.println("Rendering Square...");
  }
 }
```

```
class Circle extends Polygon {

  // renders circle
  public void render() {
    System.out.println("Rendering Circle...");
  }
}

class Main {
  public static void main(String[] args) {

    // create an object of Square
    Square s1 = new Square();
    s1.render();

    // create an object of Circle
    Circle c1 = new Circle();
    c1.render();
  }
 }
```

Interfaces in Java





Interfaces in Java

- Another way to achieve abstraction in Java, is with interfaces.
- An Interface in Java programming language is defined as an abstract type used to specify the behaviour of a class.
- An interface in Java is a blueprint of a behaviour.
- A Java interface contains static constants and abstract methods.

Key Properties of Interface:

- Like abstract classes, interfaces cannot be used to create objects
- Interface methods do not have a body - the body is provided by the "implement" class
- On implementation of an interface, you must override all of its methods
- Interface methods are by default `abstract` and `public`
- Interface attributes are by default `public`, `static` and `final`
- An interface cannot contain a constructor (as it cannot be used to create objects)

Why And When To Use Interfaces?

1) To achieve security - hide certain details and only show the important details of an object (interface).

2) Java does not support "multiple inheritance" (a class can only inherit from one superclass). However, it can be achieved with interfaces, because the class can implement multiple interfaces.

```java
// Interface
interface Animal {
  public void animalSound(); // interface method (does not have a body)
  public void sleep(); // interface method (does not have a body)
}

// Pig "implements" the Animal interface
class Pig implements Animal {
  public void animalSound() {
    // The body of animalSound() is provided here
    System.out.println("The pig says: wee wee");
  }
  public void sleep() {
    // The body of sleep() is provided here
    System.out.println("Zzz");
  }
}
```

```java
class Main {
  public static void main(String[] args) {
    Pig myPig = new Pig();  // Create a Pig object
    myPig.animalSound();
    myPig.sleep();
  }
}
```

```
The pig says: wee wee
Zzz
```

Exception handling

Exception handling:

- Fundamentals of Exception
- Exception Types,
- Using try-catch,
- Multiple try-catch clauses,
- Nested try statements,
- throw, throws,
- finally,
- Built-in Exceptions

Fundamentals of Exception

- In Java, Exception is an unwanted or unexpected event, which occurs during the execution of a program, i.e. at run time, that disrupts the normal flow of the program's instructions.
- The program can catch and handle exceptions When an exception occurs within a method, it **creates an object**.
- The object is called the **exception object**.
- It contains information about the exception, such as the **name and description** of the exception and the **state** of the program when the exception occurred.

Major reasons why an exception Occurs

- Invalid user input
- Device failure
- Loss of network connection
- Physical limitations (out-of-disk memory)
- Code errors
- Opening an unavailable file

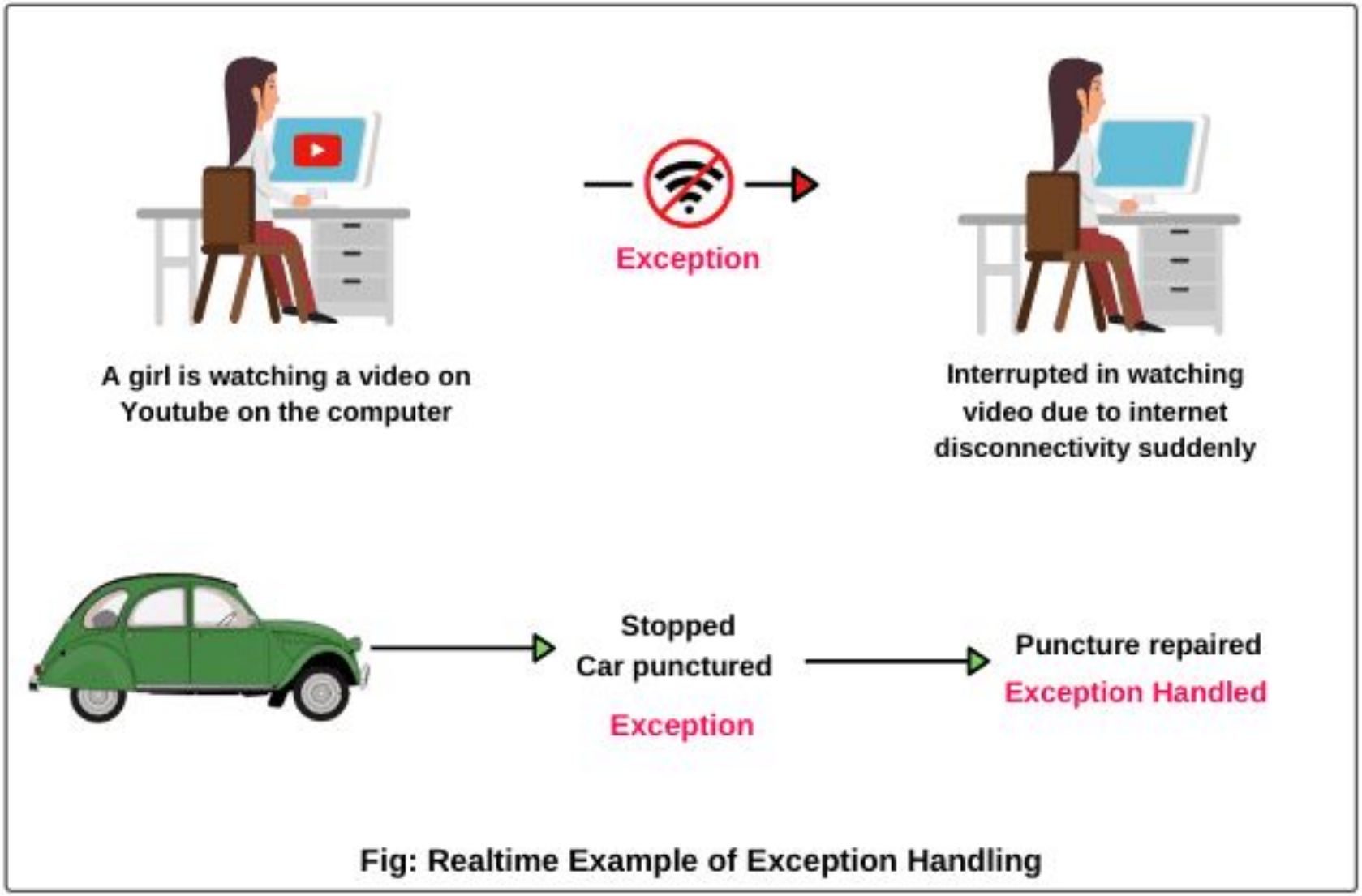


Fig: Realtime Example of Exception Handling

What are Java Errors?

- Errors in Java represent irrecoverable conditions such as the Java Virtual Machine (JVM) running out of memory, memory leaks, stack overflow errors, library incompatibility, and infinite recursion.
- These errors are typically beyond the control of the programmer and can lead to program termination.
- It's generally not advisable to handle errors programmatically, as they often indicate severe issues that require attention at a system level.

Difference between Error and Exception

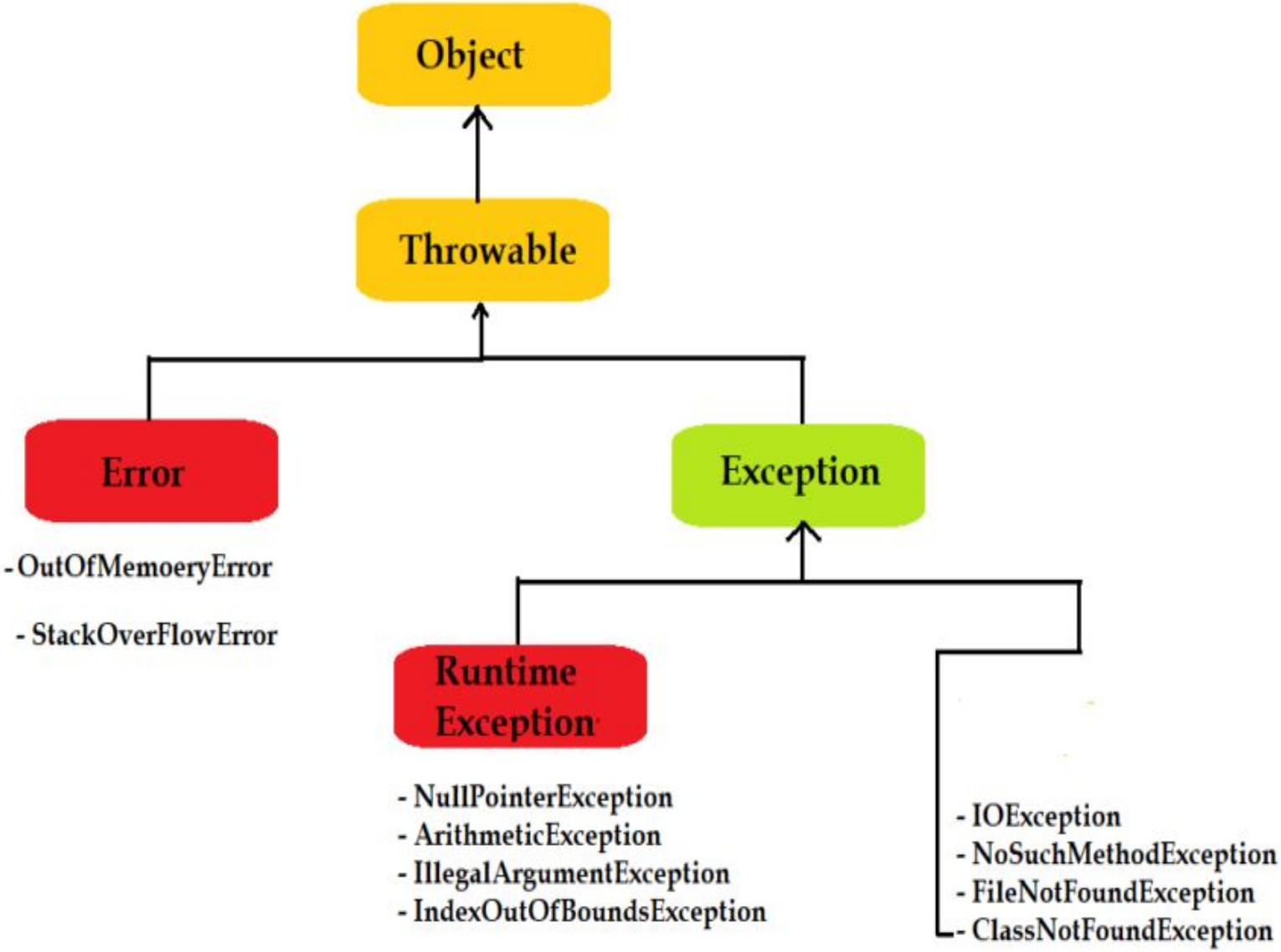
Here are the differences between Errors and Exceptions in Java:

Errors

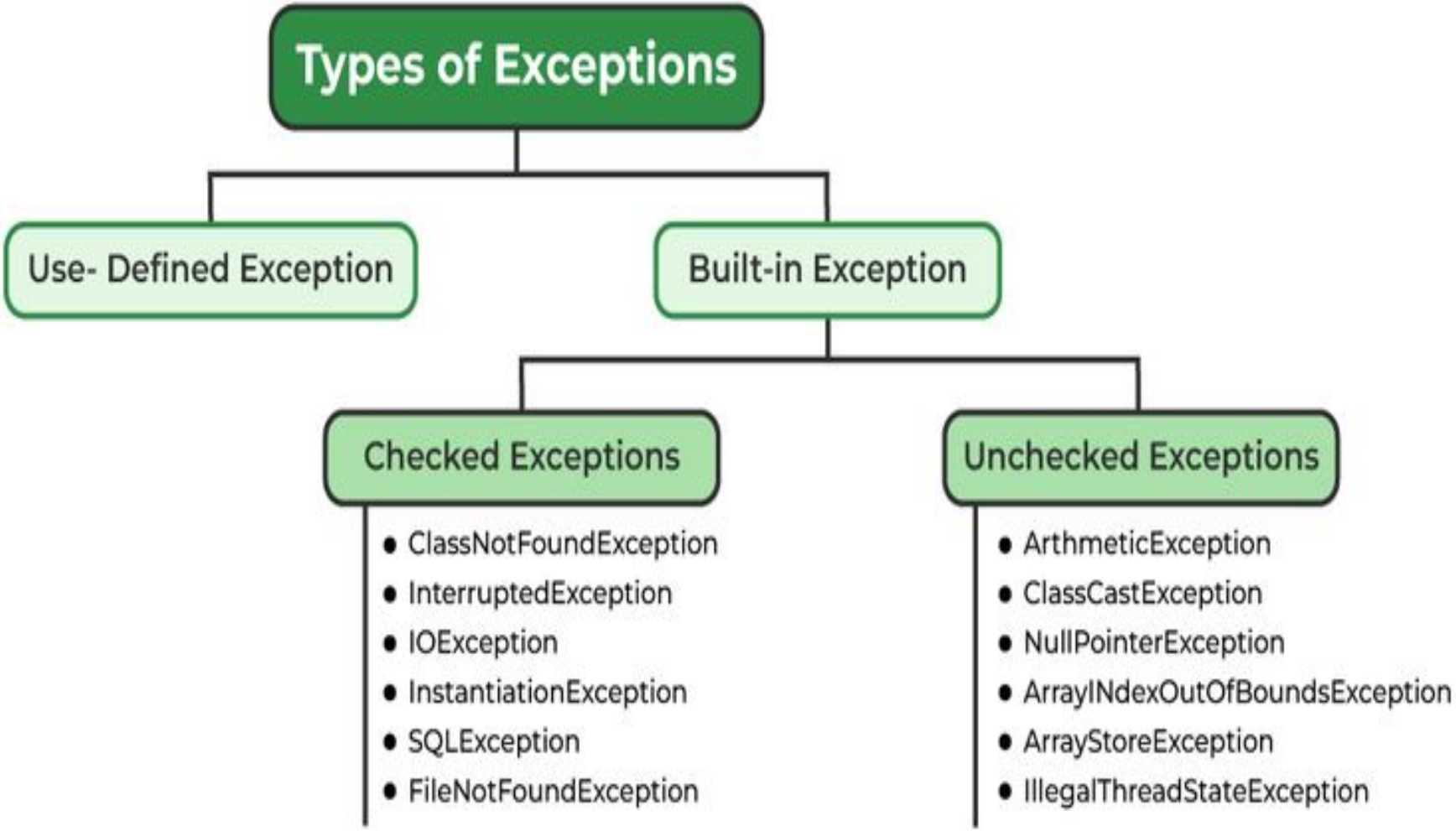
- Errors represent **serious, unrecoverable** problems that typically result from system-level issues or resource exhaustion.
- The application code generally does not handle errors, as they are beyond its control
- Examples of errors include `OutOfMemoryError`, `StackOverflowError`, and `VirtualMachineError`.
- Errors are unchecked and are subclasses of the `Error` class, which extends `Throwable`.

Exceptions

- Exceptions represent conditions that can occur during the normal execution of a program and can be handled by application code.
- Examples of exceptions include `NullPointerException`, `ArrayIndexOutOfBoundsException`, and `ArithmeticException`.
- Exceptions are subclasses of the `Exception` class, which extends `Throwable`.


Object
Throwable
Error
Exception
- OutOfMemoeryError
- StackOverFlowError
Runtime Exception
- NullPointerException
- ArithmeticException
- IllegalArgumentException
- IndexOutOfBoundsException
- IOException
- NoSuchMethodException
- FileNotFoundException
- ClassNotFoundException

Types of Exceptions:



Types of Exceptions:

1. Checked Exceptions

- **Definition:** Exceptions that are checked (verified) at **compile-time**.
- **Rule:** You must either **handle them** using try-catch or **declare them** with throws in the method signature.
- **When they occur:** Usually due to **external factors** (I/O errors, missing files, network issues, etc.) that are outside the program's control.
- **Examples:**
  - IOException
  - SQLException
  - ClassNotFoundException

```java
import java.io.*;

public class CheckedExample {
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("file.txt"); // Checked exception
        } catch (IOException e) {
            System.out.println("File not found: " + e.getMessage());
        }
    }
}
```

- If you don’t handle it, the compiler gives an **error**.

2. Unchecked Exceptions

- **Definition:** Exceptions that occur at **runtime** and are **not checked** at compile-time.
- **Rule:** You are **not forced** to handle or declare them.
- **When they occur:** Usually due to **programming errors** like logic mistakes or bad input.
- **Examples:**
  - NullPointerException
  - ArrayIndexOutOfBoundsException
  - ArithmeticException

```java
public class UncheckedExample {
    public static void main(String[] args) {
        int a = 10 / 0; // Unchecked exception
    }
}
```

The compiler will compile it fine, but it will throw an error **while running**.

Throwable

- Error (Unchecked - serious issues, usually not handled)
  - OutOfMemoryError
  - StackOverflowError
  - ...
- Exception (Mostly Checked)
  - IOException (Checked)
    - FileNotFoundException
    - ...
  - SQLException (Checked)
  - ClassNotFoundException (Checked)
  - RuntimeException (Unchecked)
    - NullPointerException
    - ArithmeticException
    - ArrayIndexOutOfBoundsException
    - ...

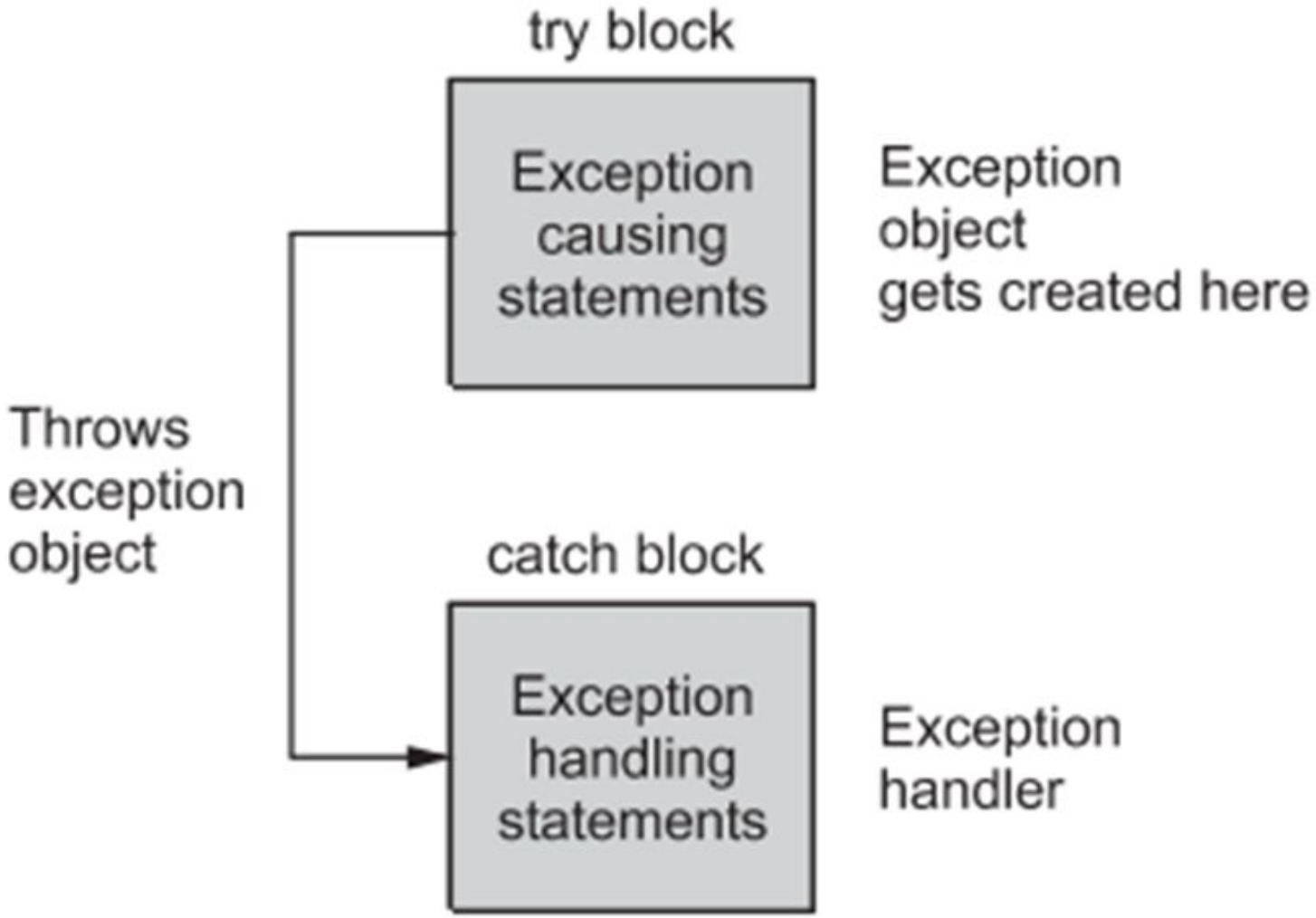
Benefits of Exception Handling:

- **Program Robustness:** Prevents abrupt program termination due to unexpected errors.
- **Maintain Normal Flow:** Allows the program to continue execution even after an error occurs, by providing a mechanism to recover or gracefully exit.
- **Error Separation:** Separates error-handling code from the main business logic, improving code readability and maintainability.

Core Components of Java Exception Handling:

- try block
- catch block
- finally block
- throw keyword.
- throws keyword

Exception handling Using try-catch



Exception handling Using try-catch

```
try {
  //  Block of code to try
}
catch(Exception e) {
  //  Block of code to handle errors
 }
```

try block:

This block encloses the code segment where an exception might occur. If an exception is thrown within this block, the remaining code in the try block is skipped, and control is transferred to the appropriate catch block.

catch block(s):

One or more catch blocks can follow a try block. Each catch block specifies the type of exception it can handle (e.g., ArithmeticException, IOException, NullPointerException).

This will generate an error, because **myNumbers[10]** does not exist.

```
public class Main {
  public static void main(String[ ] args) {
    int[] myNumbers = {1, 2, 3};
    System.out.println(myNumbers[10]); // error!
  }
}
```

The output will be something like this:

Using try-catch block:

```
public class Main {
  public static void main(String[ ] args) {
    try {
      int[] myNumbers = {1, 2, 3};
      System.out.println(myNumbers[10]);
    } catch (Exception e) {
      System.out.println("Something went wrong.");
    }
  }
}
```

The output will be:

```
Something went wrong.
```

finally Block

The `finally` statement lets you execute code, after `try...catch`, regardless of the result:

```
public class Main {
  public static void main(String[] args) {
    try {
      int[] myNumbers = {1, 2, 3};
      System.out.println(myNumbers[10]);
    } catch (Exception e) {
      System.out.println("Something went wrong.");
    } finally {
      System.out.println("The 'try catch' is finished.");
    }
  }
}
```

The output will be:

```
Something went wrong.
The 'try catch' is finished.
```

Multiple try-catch clauses

Multiple catch blocks are used when a section of code within the try block can potentially throw various types of exceptions, and each exception type requires specific handling logic.

```
try {
    // Code that might throw exceptions
} catch (ExceptionType1 e1) {
    // Handle ExceptionType1
} catch (ExceptionType2 e2) {
    // Handle ExceptionType2
} catch (ExceptionTypeN eN) {
    // Handle ExceptionTypeN
}
```

Multi-Catch Syntax (Java 7+):

Java 7 introduced a more concise way to handle multiple exceptions that require the same handling logic using a single catch block with the pipe symbol (|).

```
try {
    // Code that might throw exceptions
} catch (IOException | SQLException e) {
    // Handle both IOException and SQLException with the same logic
}
```

Nested try statements

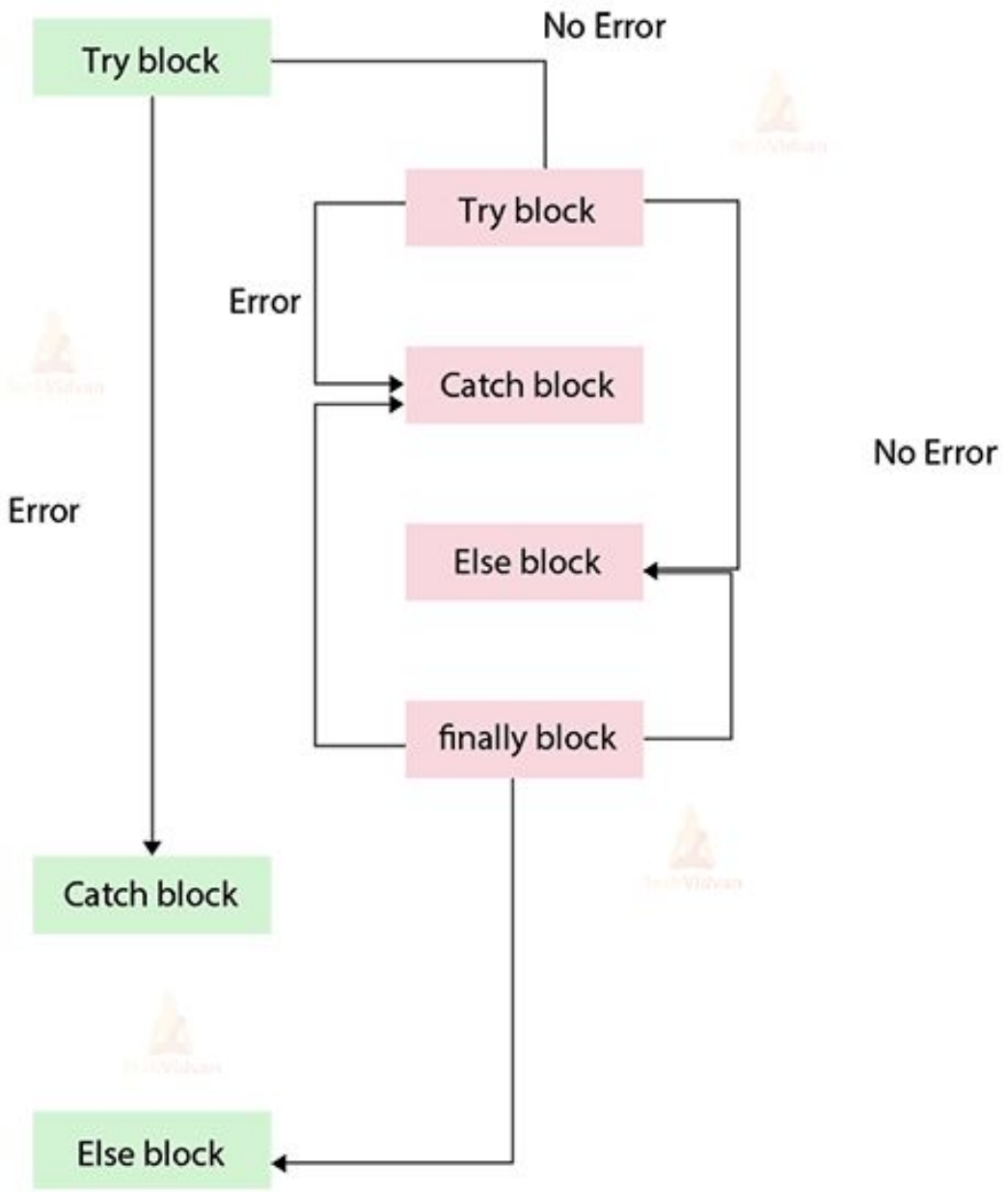
- In Java, a "nested try statement" refers to the practice of placing one or more try blocks inside another try block.
- This structure allows for more granular exception handling, where different levels of potential errors can be addressed independently.

Purpose of Nested Try Statements:

1) Handling Different Error Types
2) Hierarchical Exception Management:

```java
try {
    // Outer try block statements
    try {
        // Inner try block statements
    } catch (InnerExceptionType e) {
        // Handle inner exception
    }
    // More outer try block statements
} catch (OuterExceptionType e) {
    // Handle outer exception
}
```

Nested Try-catch flowchart:



Methods to print the Exception information in java

1. Using print Stack Trace ()

The print Stack Trace () method of the Throwable class prints the stack trace of the exception, including the class names, method names, and line numbers where the exception occurred.

Example

```
try {
    // Code that might throw an exception
} catch (Exception e) {
    e.printStackTrace();
}
```

2. Using getMessage()

The getMessage() method of the Throwable class returns a string describing the exception.

Example

```
1
2 try {
3     // Code that might throw an exception
4 } catch (Exception e) {
5     System.out.println("Exception message: " + e.getMessage());
6 }
```

3.Using toString()

The toString() method of the Throwable class returns a string representation of the exception, which typically includes the class name followed by the message.

Example

```
try {
    // Code that might throw an exception
} catch (Exception e) {
    System.out.println("Exception: " + e.toString());
}
```

THROW KEYWORD

- The throw keyword is used to create a **custom error**.
- The throw statement is used together with an exception type.
- There are many exception types available in Java: ArithmeticException,

  ClassNotFoundException,

  ArrayIndexOutOfBoundsException,

  SecurityException, etc.

Throw an exception if **age** is below 18 (print "Access denied"). If age is 18 or older, print "Access granted":

```java
public class Main {
  static void checkAge(int age) {
    if (age < 18) {
      throw new ArithmeticException("Access denied - You must be at least 18 years old.");
    }
    else {
      System.out.println("Access granted - You are old enough!");
    }
  }

  public static void main(String[] args) {
    checkAge(15); // Set age to 15 (which is below 18...)
  }
}
```

```
Exception in thread "main" java.lang.ArithmeticException: Access denied
        at Main.checkAge(Main.java:4)
        at Main.main(Main.java:12)
```

Example : 2

```
// Throwing an arithmetic exception
class Geeks {
    public static void main(String[] args){
        int numerator = 1;
        int denominator = 0;

        if (denominator == 0) {
            // Manually throw an ArithmeticException
            throw new ArithmeticException("Cannot divide by zero");
        } else {
            System.out.println(numerator / denominator);
        }
    }
}
```

THROWS KEYWORD

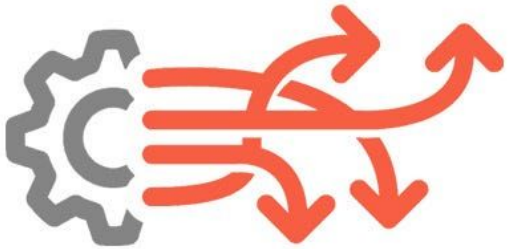
- The throws keyword in Java is used in a method's signature to declare the types of checked exceptions that the method might throw during its execution.
- **Syntax:**

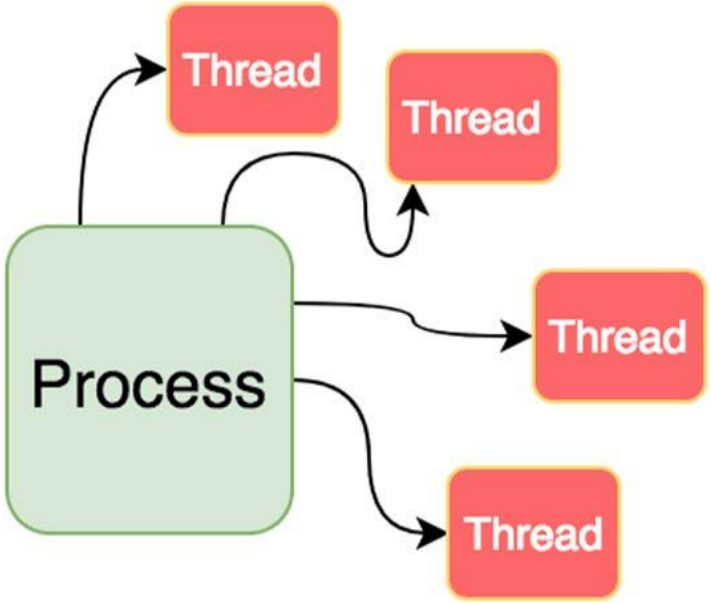
```
returnType methodName(parameters) throws ExceptionType1, ExceptionType2 {
    // method body
}
```

```java
public class Main {
  static void checkAge(int age) throws ArithmeticException {
    if (age < 18) {
      throw new ArithmeticException("Access denied - You must be at least 18 years old.");
    }
    else {
      System.out.println("Access granted - You are old enough!");
    }
  }

  public static void main(String[] args) {
    checkAge(15); // Set age to 15 (which is below 18...)
  }
}
```

Feature	`throw`	`throws`
Purpose	Explicitly throw an exception instance.	Declare exceptions a method might throw.
Syntax	Followed by an exception instance.	Followed by one or more exception class names.
Location	Inside method body/code block.	In the method signature.
Quantity	Throws one exception at a time.	Declares one or more exceptions.
Exception Type	Primarily for unchecked, but can be for checked if handled or declared.	Primarily for checked exceptions, but can declare unchecked.

MultiThreading





MultiThreading:

- Java Thread Model,
- Main Thread,
- Creating a Thread ,
- Creating Multiple Threads.

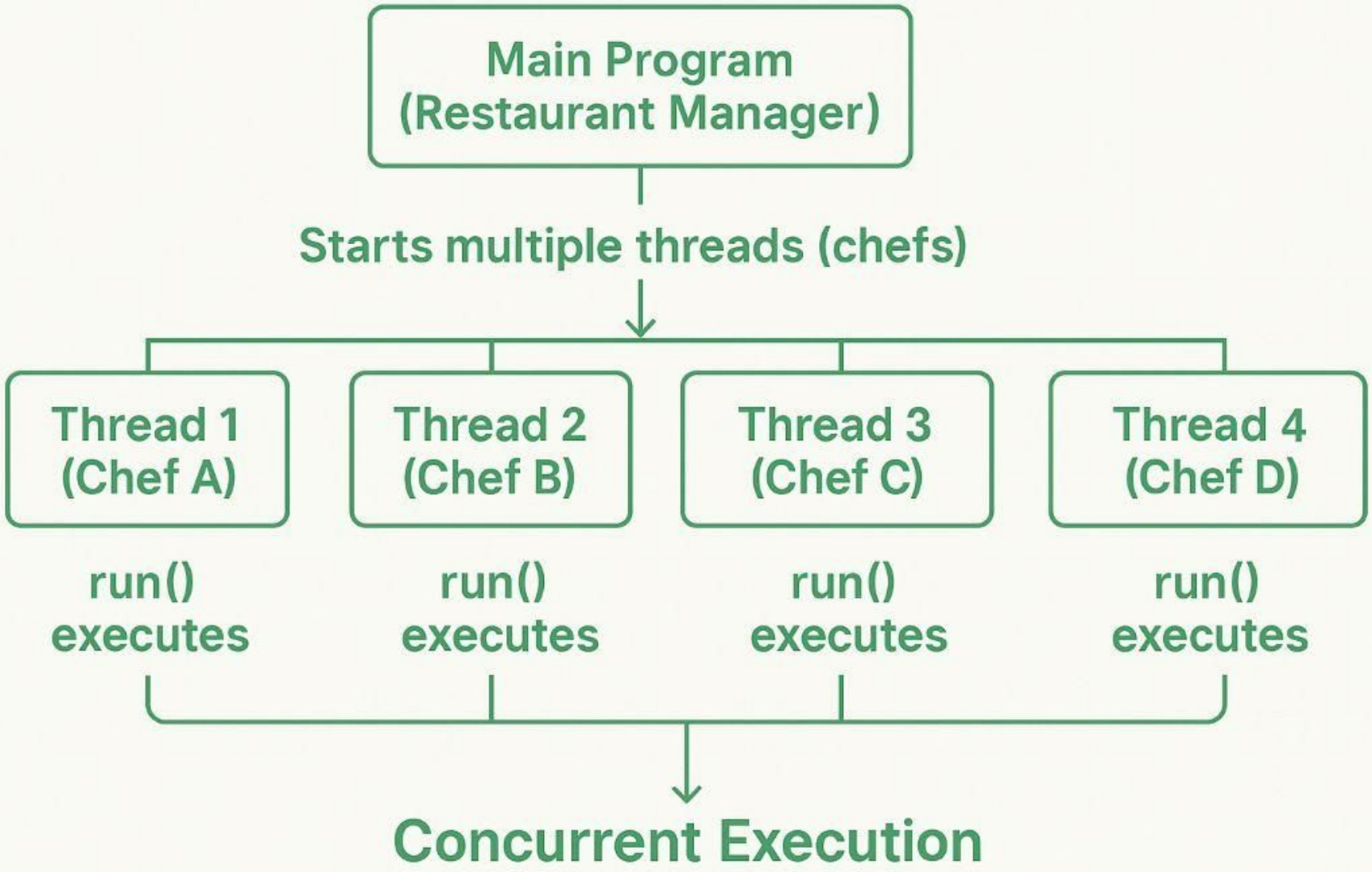
Multithreading

- **Thread:**

  A thread is a single sequential (separate) flow of control within program. Sometimes, it is called an execution context or light weight process.

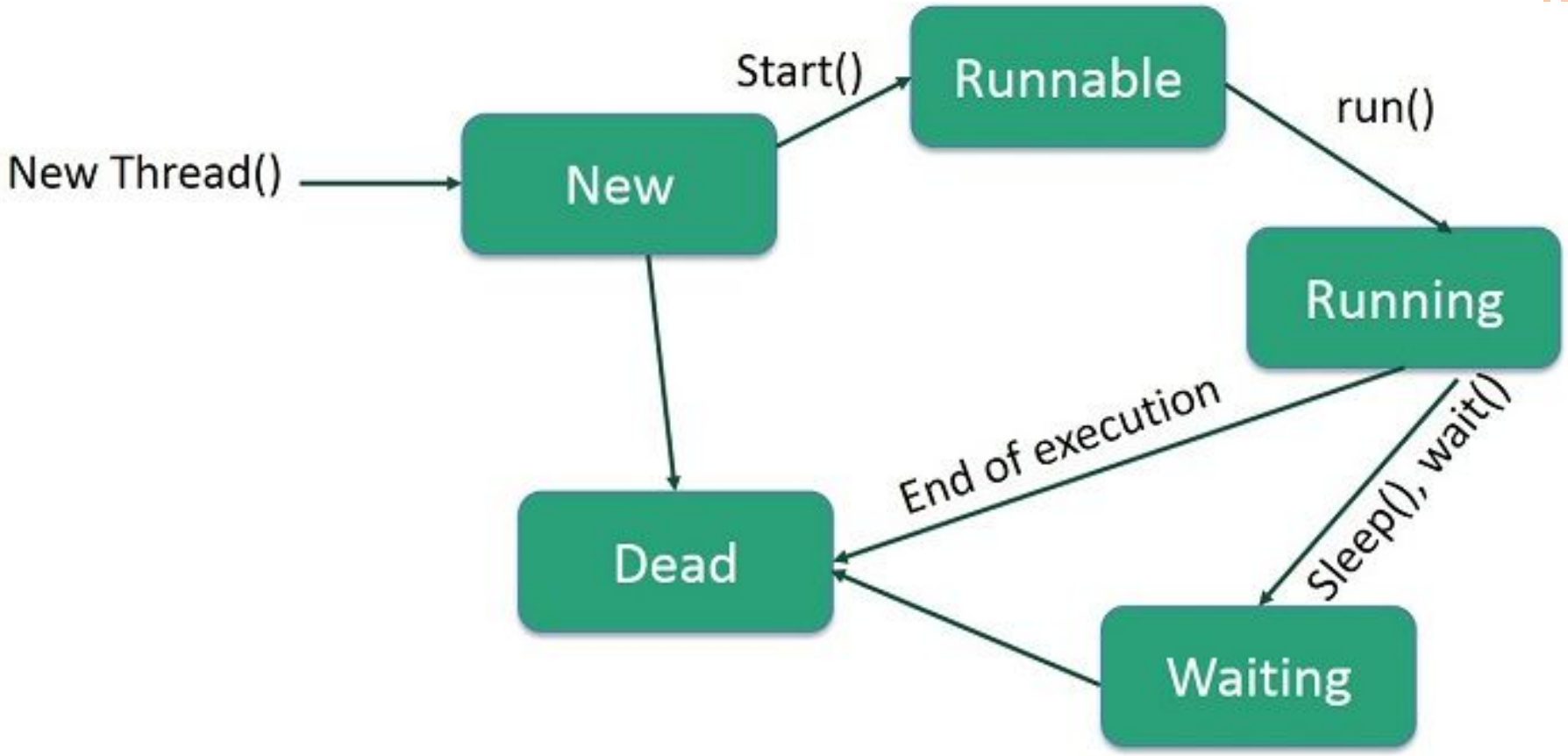
- **Multithreading:**

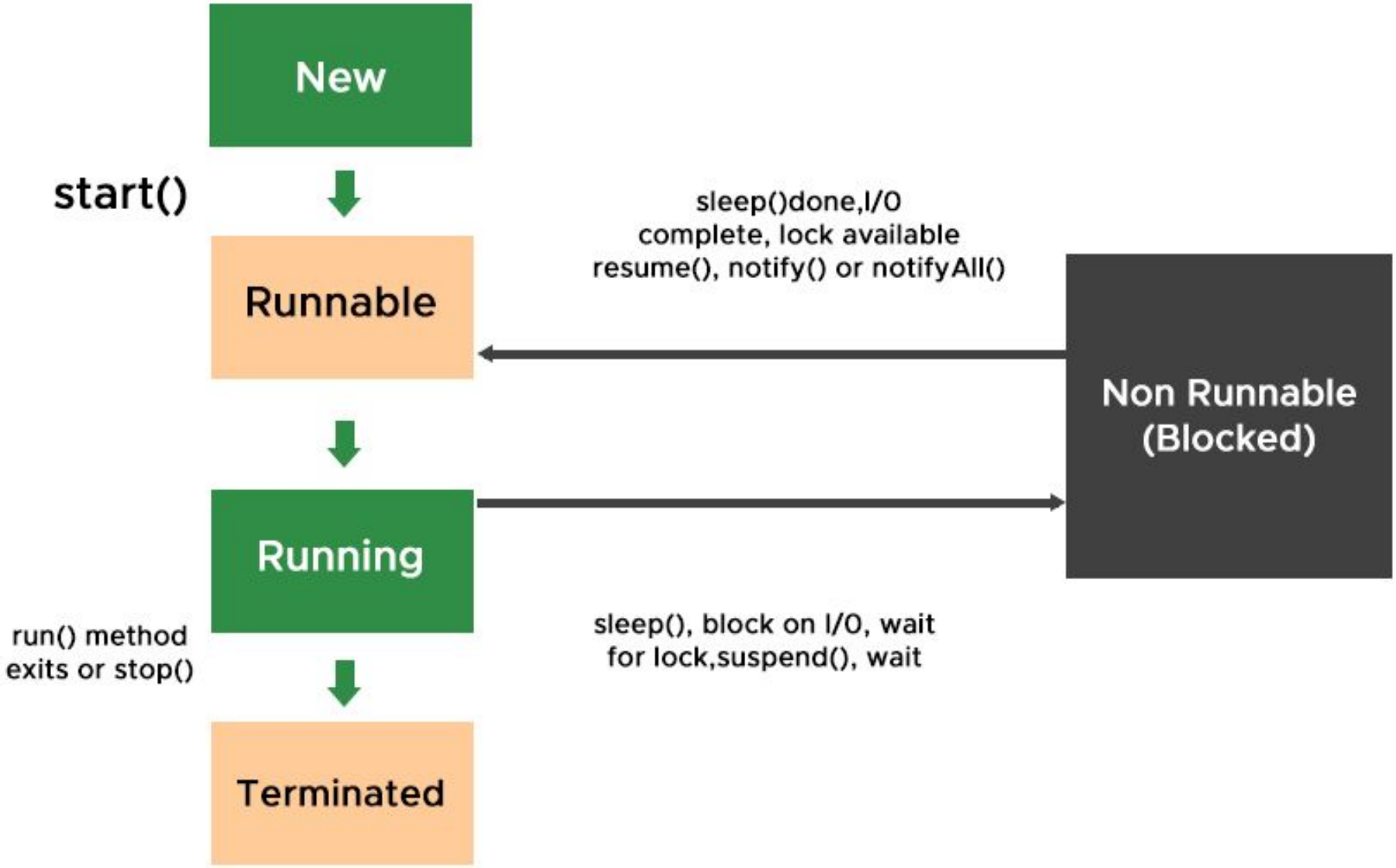
- It is a Java feature that enables the concurrent execution of two or more parts of a program, maximizing CPU utilization. Each part of such a program is called a thread.
- So, threads are lightweight processes within a process.


Main Program
(Restaurant Manager)
Starts multiple threads (chefs)
Thread 1
(Chef A)
Thread 2
(Chef B)
Thread 3
(Chef C)
Thread 4
(Chef D)
run()
executes
run()
executes
run()
executes
run()
executes
Concurrent Execution

Life Cycle of a Thread or Thread Model

- A thread goes through various stages in its life cycle.




New
start()
Runnable
sleep()done,I/O
complete, lock available
resume(), notify() or notifyAll()
Non Runnable
(Blocked)
Running
run() method
exits or stop()
sleep(), block on I/O, wait
for lock,suspend(), wait
Terminated

Following are the stages of the life cycle −

1. **New** − A new thread begins its life cycle in the new state. It remains in this state until the program starts the thread. It is also referred to as a **born thread**.

1. **Runnable** − After a newly born thread is started, the thread becomes runnable. A thread in this state is considered to be executing its task.

1. **Waiting** − Sometimes, a thread transitions to the waiting state while the thread waits for another thread to perform a task. A thread transitions back to the runnable state only when another thread signals the waiting thread to continue executing.

4. Timed Waiting – A runnable thread can enter the timed waiting state for a specified interval of time. A thread in this state transitions back to the runnable state when that time interval expires or when the event it is waiting for occurs.

5. Terminated (Dead) – A runnable thread enters the terminated state when it completes its task or otherwise terminates.

Thread Priorities

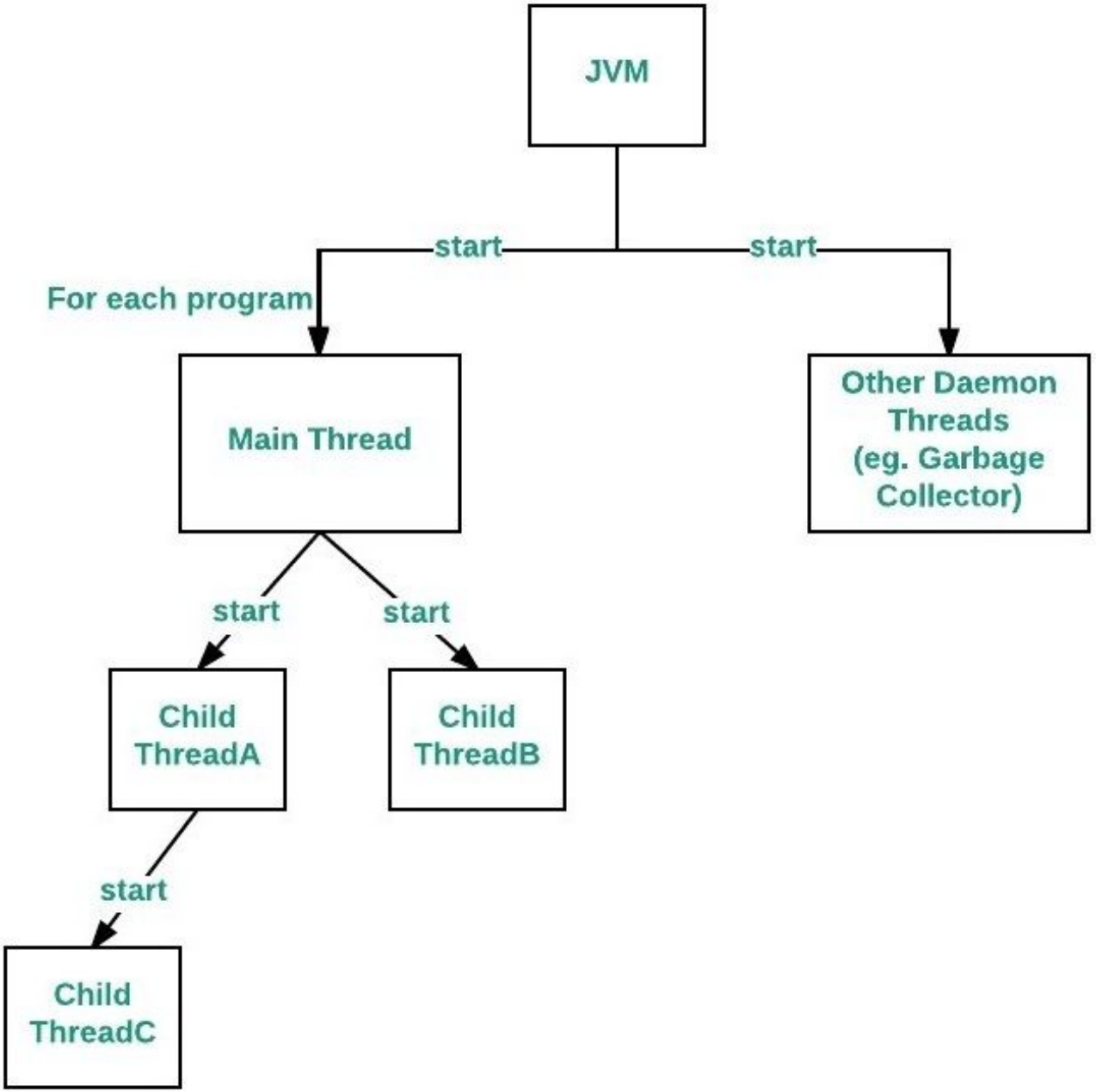
- Every Java thread has a priority that helps the operating system determine the order in which threads are scheduled.
- Java thread priorities are in the range between **MIN_PRIORITY (a constant of 1)** and
- **MAX_PRIORITY (a constant of 10).**
- By default, every thread is given priority **NORM_PRIORITY (a constant of 5).**
- Threads with higher priority are more important to a program and should be allocated processor time before lower-priority threads.
- However, thread priorities **cannot guarantee the order in which threads execute** and are very much platform dependent.

Main Thread

- The main thread in Java is the default thread that automatically starts executing when a Java program begins.
- It is the entry point for the program's execution and serves as the parent to any other threads (child threads) that might be created within the application.
- **How to control Main thread**
- The main thread is created automatically when our program is started. To control it we must obtain a reference to it.

- This can be done by calling the method **currentThread( )** which is present in Thread class.
- This method returns a reference to the thread on which it is called.
- The default **priority of Main thread is 5** and for all remaining user threads priority will be inherited from parent to child.

The flow diagram is as follows:



```java
public class TestThread {
    public void printName() {
        System.out.println("Thread Name: " +
Thread.currentThread().getName());
        System.out.println("Thread Priority: "
+Thread.currentThread().getPriority());
    }
    public static void main(String args[]) {
        TestThread thread = new TestThread();
        thread.printName();
    }
}
```

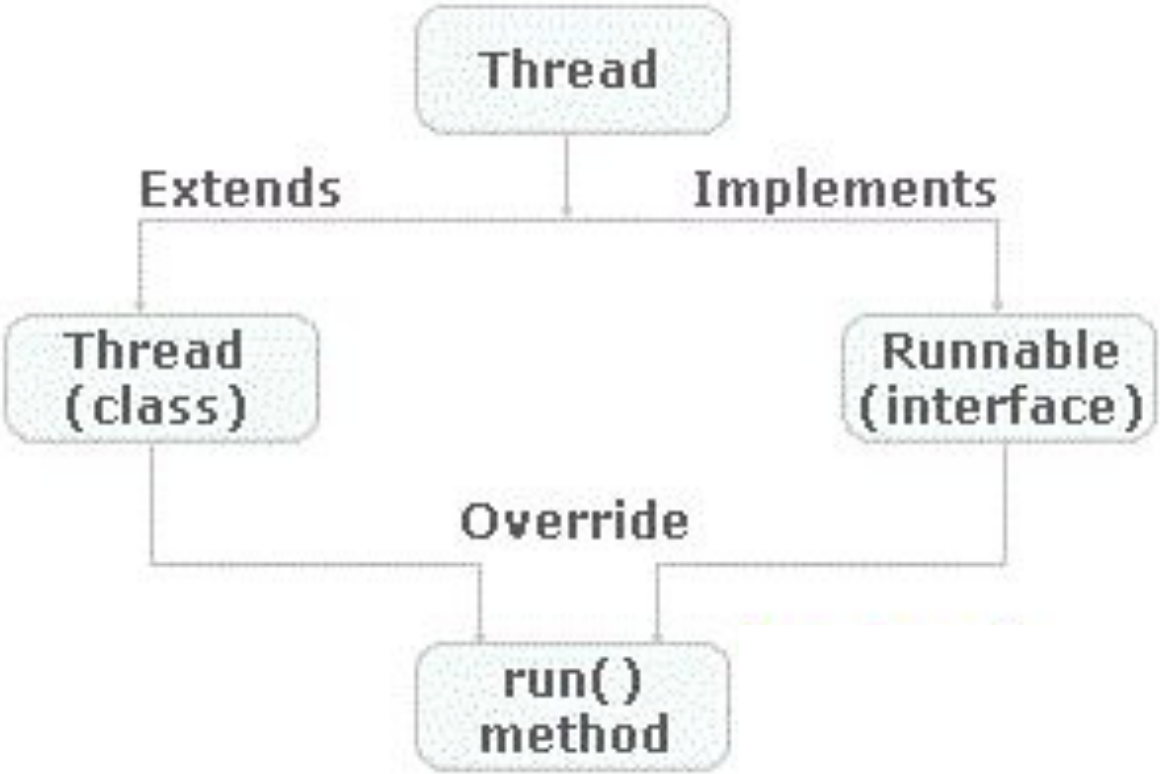
Output

```
Thread Name: main
Thread Priority: 5
```

Creating a Thread

In Java, threads can be created using two primary methods:

1. **Extending the `Thread` class**
2. **Implementing the `Runnable` interface**



1. Creating Threads by Extending the Thread class

- Create a new class that extends java.lang.Thread.
- Override the run() method within this new class. The code you want the thread to execute should be placed inside this run() method.
- In your main method or another part of your application, create an instance of your custom Thread subclass.
- Call the start() method on this instance. This will initiate the new thread of execution and cause its run() method to be invoked by the Java Virtual Machine (JVM).

```java
class MyThread extends Thread {
    public void run() {
        System.out.println("MyThread is running.");
    }
}

public class ThreadExample {
    public static void main(String[] args) {
        MyThread thread = new MyThread();
        thread.start(); // Starts the new thread
    }
}
```

2. Creating Threads by Implementing the Runnable interface

- Create a new class that implements the java.lang.Runnable interface.
- Implement the run() method required by the Runnable interface. The code for the thread's execution goes here.
- In your main method or elsewhere, create an instance of your class that implemeants Runnable.
- Create a Thread object, passing your Runnable instance to its constructor.
- Call the start() method on the Thread object.

```java
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("MyRunnable is running.");
    }
}

public class RunnableExample {
    public static void main(String[] args) {
        MyRunnable runnable = new MyRunnable();
        Thread thread = new Thread(runnable);
        thread.start(); // Starts the new thread
    }
}
```

Creating Multiple Threads

1) Extending the Thread class:

```
class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println("Thread " + Thread.currentThread().getName() + "
        // Add your thread-specific logic here
    }
}

public class ThreadExample {
    public static void main(String[] args) {
        MyThread thread1 = new MyThread();
        MyThread thread2 = new MyThread();

        thread1.start(); // Starts thread1
        thread2.start(); // Starts thread2
    }
}
```

2) Implementing the Runnable interface:

```
class MyRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("Runnable thread " + Thread.currentThread().getNa
        // Add your thread-specific logic here
    }
}

public class RunnableExample {
    public static void main(String[] args) {
        MyRunnable runnable1 = new MyRunnable();
        MyRunnable runnable2 = new MyRunnable();

        Thread thread1 = new Thread(runnable1);
        Thread thread2 = new Thread(runnable2);

        thread1.start(); // Starts thread1
        thread2.start(); // Starts thread2
    }
}
```

SYNCHRONIZATION OF MULTIPLE THREADS IN JAVA

- Synchronization in Java is a mechanism to control the access of multiple threads to shared resources, preventing data inconsistency and ensuring thread safety in multithreaded applications.
- When multiple threads concurrently access and modify the same data or code section, issues like race conditions and data corruption can arise.
- Synchronization ensures that **only one thread can access a "critical section" of code or a shared resource at a given time.**

- synchronized Keyword:

  - **Synchronized Methods:** When a method is declared synchronized, only one thread can execute that method on a given object at any time. The lock is acquired on the object instance (this).

```
public synchronized void synchronizedMethod() {
    // Critical section of code
}
```

- **Synchronized Blocks:** This allows for more granular control, synchronizing only a specific block of code within a method. The lock is acquired on the object specified in the parentheses.

```
public void someMethod() {
    synchronized (anObject) {
        // Critical section of code
    }
}
```

Basic chat Application using multithreading

```
import java.util.Scanner;

class ChatUser extends Thread {
    private String userName;
    private Scanner sc;

    ChatUser(String name) {
        this.userName = name;
        this.sc = new Scanner(System.in);
    }

    public void run() {
        for (int i = 0; i < 3; i++) { // Each user sends 3 messages
            System.out.print(userName + " says: ");
            String message = sc.nextLine();
            System.out.println(userName + " sent: " + message);
            try {
                Thread.sleep(500); // Pause for a moment
            } catch (InterruptedException e) {
                System.out.println(userName + " interrupted.");
            }
        }
        System.out.println(userName + " has left the chat.");
    }
}
```

```java
public class MultiThreadedChat {
    public static void main(String[] args) {
        ChatUser user1 = new ChatUser("User 1");
        ChatUser user2 = new ChatUser("User 2");

        user1.start();
        user2.start();

        // Check thread status
        System.out.println("User 1 alive? " + user1.isAlive());
        System.out.println("User 2 alive? " + user2.isAlive());

        // Wait for both to finish
        try {
            user1.join();
            user2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Chat ended.");
        System.out.println("User 1 alive? " + user1.isAlive());
        System.out.println("User 2 alive? " + user2.isAlive());
    }
}
```

Case Study:

Demonstrate online Banking/Library system using Inheritance, Exception handling and

Multithreading concepts

Important Questions

1. What is inheritance?Why we use inheritance? What are the types of inheritance?
2. What is superclass? What is a subclass?
3. Explain all types of inheritances?
4. Why Java doesn't support multiple inheritance?
5. What is the difference between multiple and multilevel inheritance?
6. What are the limitations of inheritance?
7. Differentiate between Abstract Classes and Interfaces with examples.(5 Marks)
8. Write a Java program that demonstrates multilevel inheritance.(4 Marks)
9. Explain try-catch-finally with an example. What happens if an exception is not caught?

Important Questions

1. What is polymorphism? Differentiate between compile-time and runtime polymorphism. [4]
2. Define the term "Built-in exceptions". List any four common Java exceptions. [4]
3. Write a Java program to create a user-defined exception for age verification. [4]
4. Explain nested try-catch block with a simple program. [6]
5.

Thank You!!!